

HUMBOLDT-UNIVERSITÄT ZU BERLIN
MATHEMATISCH-NATURWISSENSCHAFTLICHE FAKULTÄT
INSTITUT FÜR INFORMATIK

Extending Escrow Transactions for Heterogeneous Resource Management

Masterarbeit

zur Erlangung des akademischen Grades
Master of Science (M. Sc.)

eingereicht von: Dominik Bernhardt
geboren am: 04.01.1997
geboren in: Neu-Ulm

Gutachter/innen: Prof. Dr. Matthias Weidlich
Prof. Dr. Jan Mendling

eingereicht am:

This thesis presents the design, formal foundation, and implementation of a journal-based Resource Planning System that extends classical escrow transactions from aggregate counters to individually tracked resources. Typed capacity counters preserve lock-free pool admission while an append-only allocation journal records every lifecycle transition of every resource instance. Compensation, timeout handling, and saga-style recovery are relocated into the database core through paired forward and inverse procedures. An initial workflow integration on Oracle APEX Workflow and Transactional Event Queues demonstrates how process designers invoke the resource layer while error handling and concurrency remain at the database boundary. Airline seat allocation serves as the reference domain. The architecture is domain-neutral and accommodates other domains through configuration rather than code changes.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	3
1.3	Scope Delimitation	3
1.4	Approach	4
1.5	Research Contributions	5
1.6	Thesis Structure	6
2	Related Work	7
2.1	Shared pools and Escrow Transactions	8
2.2	Long-Running Transactions, Sagas, and Compensation	9
2.3	Workflow Systems and Resource Patterns	9
2.4	Immutable Logs, and Audit Trails	10
2.5	Prior Work and Synthesis	11
3	Formal Foundations	13
3.1	Notation	13
3.2	Resource Model	13
3.3	Temporal Model	17
3.4	Request Model and Lifecycle	19
3.5	Constraint System	21
3.6	Escrow Semantics on Capacity Counters	24
3.7	Compensation Model	26
3.8	Limitations of the Model	28
4	System Design	29
4.1	Escrow and Journal as Cooperating Layers	29
4.2	System Architecture	29
4.3	Domain Language	30
4.4	Data Model Design	31
4.5	Journal-Based State Management	34
4.6	Capacity Management	36
4.7	Transaction and Commit Strategy	37
4.8	Compensation Framework	39
4.9	Type Substitution System	42
4.10	Validation	43
5	Workflow Integration	45
5.1	Integration at the Process Boundary	45
5.2	Implications for Workflow Modelling	46
5.3	Integration Constraints Imposed by the Workflow Runtime	47
5.4	Workflow-Coupled Resource Execution Model	48

5.5	Reference Workflow: Flight Booking	49
5.6	Reference Workflow: Inviting Workers to Meetings	51
5.7	Plugin Design and Behavior	52
5.8	Summary	54
6	Future Work	55
6.1	Scope and Framing	55
6.2	Research Directions and Extensions	55
7	Conclusion	58
7.1	Summary of Contributions	58
7.2	Limitations	59
7.3	Reflections	59
	Acknowledgements	61
	Appendix	iii
	Mathematical Notation	iii
	Implementation Reference	v
	Code and Data Availability	v
	Core Schema Listing	v
	Schema-to-Formal-Model Mapping	vii
	Public API Surface	vii
	Implementation Pseudocode	ix
	Application Error Catalogue	xiii
	Validation Scenario Index	xiv
	AI Usage Disclosure	xvi

1 Introduction

1.1 Motivation

Business workflows fundamentally depend on resource management. Whether scheduling airline seats, booking conference rooms, or allocating cloud computing instances, workflows must reserve resources, claim them when needed and release them when no longer required. Process engines coordinate *when* steps run, but they do not natively model *which* resource instance is bound, in *what* lifecycle state, or how failures are undone.

Database systems have long supported resource management through *escrow transactions* (O’Neil, 1986), which enable high-concurrency access to aggregate resource pools. In an escrow model, resources are tracked as counters, e.g. a pool of 100 available seats changes to 95 after five reservations, with each reservation held in a session-local escrow buffer that merges into the committed counter on confirm or is discarded on rollback, without blocking concurrent reservations on a row lock. This approach excels for *homogeneous* resources where individual units are interchangeable and only the total count matters.

Business Process Management (BPM) and workflow tools are built for control flow, not for individual resource lifecycles, temporal overlap rules, or systematic compensation. Together with classical escrow, this leaves a gap for *heterogeneous* resources with unique identities, varying properties and complex lifecycles. Two short examples illustrate the gap:

- **Airline seat allocation** (the referenced use-case used throughout this thesis). Seat 14A (business, window, extra legroom) is not interchangeable with seat 32C (economy, middle, standard); passengers care *which* seat they are assigned, and the seat progresses through various states, each with distinct business rules. An aggregate counter of “available seats” cannot represent either the identity or the state.
- **Meeting-room or shared-equipment booking.** A room, piece of equipment or worker is identified by name or serial number and is booked *for a time interval*. Whether that resource is free at 10:00 AM on Monday depends on the union of all bookings that overlap that interval. That is a question about a specific instance, its lifecycle, and temporal overlap. An aggregate counter cannot answer it. A capacity total for one vehicle or one meeting is not enough either, because the same resource can be shared across unrelated contexts.

The same limitation applies to other inventory systems that track individually identifiable instances rather than interchangeable quantities.

Therefore organizations traditionally implement resource semantics in *application code*: custom services, workflow scripts, and integration layers between the process engine and the database. This split creates several problems:

- **Split resource state:** Allocation truth is scattered across workflow variables, application services, and database tables, with no single authoritative model of who holds which instance in which state and no durable, queryable history of lifecycle transitions.
- **Weak transactional guarantees:** Application and database can diverge when a workflow step commits locally but a related database update fails, or vice versa—leaving orphaned reservations, duplicate bookings, or capacity counters out of sync with instance-level state.
- **Duplicated recovery logic:** Each workflow reimplements timeout handling, cancellation, and partial-failure rollback. Compensation rules differ for each process and are difficult to verify.

Building on preliminary work (Bernhardt et al., 2024) that established the foundational layer for resource planning systems (data structure, resource lifecycles, procedures), we address these problems by shifting resource semantics into the database: resources become first-class objects with type hierarchies, temporal context and lifecycle models managed directly by the database rather than external applications.

We pursue this goal by *extending* the escrow transaction model rather than replacing it. Its strength is lock-free reservation of fungible capacity under high concurrency. However, escrow alone cannot answer which instance was taken, how to substitute when the preferred type is unavailable, which transitions are legal, or how to recover when a process aborts. We therefore *preserve* typed capacity counters with the original escrow admission semantics and *add* a cooperating lifecycle layer at instance level that covers everything escrow alone cannot express.

The two mechanisms cooperate. Counters track how many units of a type may still be claimed in a context under concurrent load, while the journal records which instance is bound to whom, its lifecycle state, and the inverse entries that restore consistency after a failure or timeout. They remain separate database artifacts in this work.

The architecture is deliberately *domain-neutral*. Resource types, allocation contexts, state machines, substitution rules and timeout policies are configurable by each deployment. We chose flight seat allocation as the use-case because it stress-tests that neutrality in one demanding setting: passengers care about *which* seat and cabin class (with a substitutable type hierarchy), each flight defines a strictly bounded temporal context with finite pooled capacity, and the passenger lifecycle spans multiple states that must stay consistent with workflow-driven holds, timeouts and compensation. The airline scenario is a concentrated proof that the extended model can carry operational complexity while remaining structurally the same for hotel rooms, equipment loans, meeting-room booking or any other domain that allocates named resources for a time interval.

In the resource core, admission rules, overlap checks, escrow buffering, timeout cancellation, and saga-style recovery replace per-workflow rollback logic, leaving process designers a small, stable procedure surface rather than bespoke cleanup in process code.

1.2 Problem Statement

This thesis addresses the following core challenge:

How can database systems extend classical escrow mechanisms to support heterogeneous resource management with individual state tracking enforced at the database boundary, while maintaining transactional consistency and enabling integration with business workflows?

Answering this requires more than richer schemas: resource admission, lifecycle enforcement, and failure recovery must shift from workflow implementation into the database boundary, so that process code orchestrates *when* actions occur while the resource core decides *what* allocations are legal and how they are undone.

This challenge decomposes into four sub-problems:

P1: Individual Resource Tracking. Heterogeneous resources require per-instance identity and history, not only aggregate pool counts. The system must record who holds which instance in which state, consistently with escrow-based capacity, without pushing semantics into every workflow.

P2: Complex State Lifecycles. Resources follow domain-specific lifecycles with distinct forward and compensating transitions. Valid state machines must be configurable per deployment and enforced in the database layer rather than re-encoded in process definitions.

P3: Compensation and Saga Patterns. Long-running processes cannot hold locks indefinitely; timeouts, cancellations and partial failures must trigger systematic recovery. Forward operations need paired inverses and journal-backed compensation so rollback and expiry are not pushed to ad hoc workflow cleanup.

P4: Workflow Integration. Business processes must invoke the resource layer without reimplementing admission, concurrency or error handling. Process designers call a small plugin surface. The resource package enforces legality, holds, timeouts, and compensation at the database boundary, including the asynchronous reservation hold window.

1.3 Scope Delimitation

In Scope:

- **Individual resource instance tracking** for heterogeneous resources with unique properties
- **State lifecycle management** supporting domain-configurable state machines

- **Immutable journal-based audit trail** providing complete traceability of all state changes
- **Temporal conflict detection** for shared resources across multiple contexts
- **Type substitution mechanisms** when preferred resources are not available
- **Systematic recovery.** The focus is on shifting error handling, concurrency and compensation into the database layer rather than exhaustive workflow coverage.
- **Initial workflow integration** demonstrating how process designers interact with the resource layer, demonstrated on Oracle APEX Workflow and Transactional Event Queues (TxEventQ) for the reservation hold window.

Out of Scope: The following extensions are discussed in Section 6 but not implemented in this thesis:

- **Automatic delay propagation and cascading rescheduling**—disruptions in one allocation context (e.g. a delayed flight) automatically forcing updates or compensation in dependent downstream bookings
- **Event notification and subscription layer**—asynchronous publication of journal-derived events so workflows or services can react (e.g. short-notice unavailability)
- **Nested and distributed sagas**—coordinated recovery across multiple resource domains or contexts (e.g. a multi-leg trip: outbound flight, event, hotel, return flight) with linked compensation

This thesis delivers a formal model and an executable *implementation* on Oracle Database. The goal is *feasibility*, not a production system: we demonstrate that the design can be realised end-to-end on a representative domain, without claiming completeness, performance, or operational hardening.

1.4 Approach

Methodologically, the work follows a design-science approach (Hevner et al., 2004). The contribution is a constructed artifact, namely a formal model together with its database realisation, rather than an empirical study or a controlled experiment. The artifact is developed iteratively from the problem analysis above and evaluated against representative allocation scenarios (Section 4.10) that exercise its core invariants. This framing also sets the standard of evidence for the rest of the thesis. Claims rest on the formal definitions of Section 3 and on qualitative scenario validation of the implementation, rather than on performance benchmarks.

1.5 Research Contributions

This thesis makes four primary contributions:

C1: Formal Model for Extended Escrow and Per-Instance Lifecycles. We develop a mathematical framework (Section 3) that extends classical escrow to heterogeneous, individually tracked resources without replacing its lock-free admission semantics on typed capacity counters. The model unifies aggregate pool management with per-instance lifecycle bindings under allocation contexts, substitution hierarchies, and domain-configurable state machines. It states explicit invariants—capacity, exclusivity, type consistency, and temporal overlap—and specifies how journal-derived active bindings and committed counter bases must align during and after reservation holds.

C2: Temporal Allocation Within Contexts. Resources are allocated inside scheduled *allocation contexts* (events, flights, time intervals). The same instance may hold independent bindings across distinct contexts. Seat 14A can be taken on a future flight while remaining free for admission on another. Availability follows from the journal and interval overlap rules (Sections 3.3 and 4.4). That moves realistic resource planning to the database layer rather than into workflow variables. Whether an instance may be admitted at all depends on its intrinsic state, which is only checked at the time of admission. Scheduled withholding of instances is managed through maintenance contexts and the **blocked** journal state (Section 3.5).

C3: Append-Only Journal and Systematic Compensation. Instead of overwriting prior allocation state, all state changes are recorded as immutable journal entries (Section 4.5). Capacity counters and journal entries are kept consistent through the transaction strategy (Section 4.7): session-local escrow buffers hold capacity changes until merge or discard, while journal rows record per-instance intent and lifecycle progress. Each state-changing procedure is paired with an inverse procedure (Section 4.8). Transaction failures or timeouts invoke compensating journal entries deterministically instead of ad hoc cleanup in workflow or application code. This is where the responsibility shift materialises in the implementation: correctness and recovery are guaranteed by the resource package, not by each process designer.

C4: Initial Workflow Integration. Section 5 provides an initial workflow integration on Oracle APEX Workflow, not a comprehensive production solution. Its focus is the asynchronous reservation hold window, where Transactional Event Queues carry human confirm or cancel decision to the database session that opened a reservation. Workflow designers declare simple intents on resources through a custom plugin. Admission, journal updates, timeout handling, and compensation remain at the database boundary. Additional lifecycle steps such as check-in, boarding, and complete illustrate how domain-configurable states can extend the same pattern. In our implementation they are kept as simple synchronous calls rather than further queue-backed paths.

1.6 Thesis Structure

Section 2 positions this work within the broader literature, related research and current studies in the field. Section 3 develops the formal model. Section 4 translates the formal model into a concrete database architecture. It closes with manual validation of core invariants. Section 5 demonstrates connecting the resource package to Oracle APEX Workflow through a custom plugin, using TxEventQ for the asynchronous reservation hold window. Section 6 discusses limitations and possibilities for further contributions. Section 7 closes the thesis. The appendix covers implementation reference material, API summary, and the validation scenario index.

2 Related Work

Previous research relevant to the *database-level transactional semantics* can generally be categorised within four topics: lock-free aggregate resource management, compensation for long-running processes, append-only state recording, and workflow and resource integration.

The Problem: Application-Layer Resource Coordination. We established that heterogeneous resource management exceeds what classical escrow and workflow engines provide natively. In practice, those limits push developers toward application-layer coordination: long-lived holds, domain-specific interchangeability, and hand-coded recovery logic.

Ad hoc transactions. Lu et al. (2023) define *ad hoc transactions* as transactions coordinated by application code rather than by the database system itself. In this pattern, developers must manually implement locking, validation checks, error handling, and a compensation strategy for every operation. Resource reservation, holding windows, time-bound cancellation, and compensation are typical examples.

A recent survey of 91 such transactions across eight popular open-source web applications illustrates how common and risky the pattern is in practice. Two findings are especially relevant:

1. Each application uses ad hoc transactions (up to 16 per application), with 71 identified as critical in business logic such as checkout, payment and booking.
2. 53 of the 91 studied instances exhibit reproducible correctness issues.

That significant error rate suggests that complex resource lifecycles are unlikely to be safer when managed outside the database boundary.

A traditional flight booking system might implement:

- A locking mechanism to prevent double-booking.
- Validation logic prior to updates or inserts.
- Custom timeout logic to release abandoned transactions.
- Compensation logic to undo failed bookings.

These procedures are typically hand-coded and are written or tested in isolation and are prone to subtle race conditions. We take the opposite approach: rather than relying on separate validation, locking, error handling, and compensation logic using ad hoc transactions, we extend the database’s capabilities to coordinate heterogeneous resource management directly, keeping allocation state authoritative in one place rather

than distributed across workflow variables, application services, and ad hoc database updates.

2.1 Shared pools and Escrow Transactions

Database systems have long supported high-concurrency reservation on shared pools through *escrow transactions* (O’Neil, 1986), a technique whose roots lie in the hot-field concurrency control of IMS/VS Fast Path (Gawlick and Kinkade, 1985). Concurrent transactions adjust a scalar counter through session-local escrow buffers, avoiding row-level locks on the hot aggregate while still preventing overbooking.

Escrow implementations differ in when they enforce the pool limit. An optimistic scheme validates only when session buffers merge into the committed counter at commit. A transaction may run further and fail only at the end. A pessimistic scheme checks each buffer extension as it is applied, against the committed base and other sessions’ in-flight buffers. Oracle’s *RESERVABLE* columns (Oracle Corporation, 2026b) use the pessimistic model. Our concept therefore follows the same rule (Section 3.6). That timing works in our favour. A pool rejection happens before any autonomous journal write, so fewer failed attempts leave instance-level rows that must be compensated later.

Example. Consider a flight with 100 available seats. Without escrow, each booking would lock the *available seats* counter, serializing all concurrent bookings. With escrow, each session holds its change in an escrow buffer and the database validates that the aggregate stays within the specified constraint (e.g. $\text{counter} \geq 0$). Multiple concurrent sessions can proceed without blocking each other.

Limitation. Escrow excels at homogeneous capacity, when all instances are interchangeable. However, escrow does not address the full challenge of heterogeneous resource management:

- **No instance tracking.** Escrow records how many seats are taken, but not *which* seats.
- **No lifecycle states.** A resource progresses through various states—escrow counters cannot represent that.
- **No domain-specific ruleset.** Which state transitions are allowed?

These concerns are typically pushed into application or workflow code, outside the transactional guarantees that motivate escrow in the first place. We preserve the escrow semantics for counter constraints while adding an adjacent allocation journal that tracks every instance-level lifecycle transition.

2.2 Long-Running Transactions, Sagas, and Compensation

When resource operations span multiple steps or require external confirmation, they are considered long-running transactions. A resource reservation might wait multiple minutes for payment confirmation, or a multi-leg seat booking might span multiple services. Holding classical ACID locks for such durations is impractical, which is why the transaction-processing literature turns to compensation-based recovery for long-lived activities (Gray and Reuter, 1993).

The saga model of Garcia-Molina and Salem (1987) demonstrates the standard recovery strategy: forward steps commit independently, and failures trigger compensating transactions, reversing the effects.

Example. A flight booking saga might have three distinct steps:

- Reserve Seat (compensation: release seat)
- Payment (compensation: refund)
- Issue boarding pass (compensation: deactivate boarding pass)

Depending on the step, the saga triggers reverse transactions to keep the system in a consistent state.

Saga semantics are typically pushed to application code or workflows. Workflow engines expose generic timers and error handlers, but they do not define domain-specific reservation steps, the inverse and recovery logic is handled per workflow. Reconciling such long-running, compensable activities with transactional guarantees is a recognised problem in the advanced-transaction-model literature for workflow contexts (Alonso et al., 1996).

We propose to relocate the compensation logic within the database boundary. Each forward procedure is paired with an inverse that appends a compensating journal entry, reversing the effects. Developers do not need to implement complex compensation logic for each operation, they just need to declare an intent with simple API calls and react to the outcome of the procedure.

2.3 Workflow Systems and Resource Patterns

Business process management coordinates *who* performs *which* tasks and in *what* order. Dumas et al. (2018) summarise that typically a *resource* is an organisational entity: a role, or a user assigned to an activity.

BPMN 2.0 (Object Management Group, 2011) is the dominant standard for modelling such processes. It specifies control flow, gateways, events, and task assignment, including resource-related attributes such as performers and potential owners. For workflow

execution, that level of resource modelling is often sufficient: the engine needs to know which role may claim a task. It does not need to govern how a finite pool of bookable assets behaves over time.

Example. In a BPM model, a task might be assigned to a role. The engine can route work to available agents, but it does not treat a seat, room, or equipment item as a bookable object with capacity limits, a schedule, and its own lifecycle. The engine does not know that such a resource is *bookable*, that only one client may hold it in an overlapping interval, or that a tentative hold must later be confirmed, timed out, or compensated.

BPMN therefore leaves the reservation patterns central to this thesis without execution semantics for:

- hold windows between reservation, confirmation, and cancellation,
- timeout-driven release,
- compensation of partially completed allocations.

A recent review by Pufahl et al. (2025) analyses 61 resource-allocation approaches in business process management, covering publications from 1995 to 2023. The study shows:

- Most approaches only show 1:1 allocation cardinalities.
- Focus is on process-oriented goals (e.g. cost minimization).
- Transactional correctness of resource operations receives limited attention.

Allocation decisions are treated as instantaneous assignments rather than stateful bindings that must be compensated if unsuccessful. Workflows identify *who* does *what*, but not *how* resource allocations are maintained transactionally. We provide the missing layer: the workflow chooses *when* to reserve; the database decides whether the reservation is allowed and how to compensate if it fails.

2.4 Immutable Logs, and Audit Trails

A recurring theme in distributed systems is the value of recording state changes as append-only log entries rather than updating previous states.

Helland (2015) argues that immutability simplifies the reasoning of distributed states: past entries never change, and compensation is the addition of an inverse entry, rather than deleting or modifying previous entries. These principles support event sourcing (Fowler, 2005), where arbitrary or current states are derived by replaying a log of events on top of a selected snapshot.

Our allocation journal adopts the *append-only* principle. Every state transition is recorded as a new entry in the journal and prior entries are never modified or deleted. This provides a complete audit trail of resource allocations without blocking concurrent sessions trying to allocate the same resource for another allocation period.

Remark. While we adopt the append-only logging, we do not implement event sourcing. There is no replay mechanism to reconstruct states based on the log. The journal serves primarily for compensation, and traceability. We combine an immutable journal with escrow semantics and lifecycle state machines—a narrower, resource-specific application of the append-only principle.

2.5 Prior Work and Synthesis

Table 1 summarises how this thesis composes prior lines of work into a single architecture. No single cited publication addresses heterogeneous resource management with concurrent pool admission, temporal allocation for multiple contexts, individual lifecycle, automatic compensation, and executable workflow alignment. The contribution is the integration of these concerns around escrow and an instance level journal.

Table 1: Positioning summary: typical prior work vs. this thesis

Concern	Typical Prior Work	This Thesis
Pool concurrency	Escrow / aggregate counters (O’Neil, 1986)	Typed escrow counters on pools (Oracle’s lock-free reservation)
Temporal allocation	Instantaneous assignment; BPM focus on 1:1 cardinality (Pufahl et al., 2025; Russell et al., 2005)	Allocation contexts; journal-derived current state; overlap rules in DB
Instance lifecycle & audit	Ad hoc app/workflow logic; mutable counters (Lu et al., 2023; Helland, 2015)	Append-only journal + configurable state machine in DB
Failure recovery	Process-level sagas (Garcia-Molina and Salem, 1987; Gray and Reuter, 1993)	Procedure-inverse pairs in resource core
Workflow boundary	Roles / external APIs (Object Management Group, 2011; Russell et al., 2005)	Workflow declares <i>when</i> ; DB enforces legality and recovery

The result is a system where correctness guarantees, like admission control, lifecycle enforcement, audit, and recovery are relocated from application code into the database boundary. Workflow designers call simple procedures and the database enforces the semantics and recovery logic. The immediate predecessor to this thesis is a study project (Bernhardt et al., 2024) that established core schema, lifecycle procedures, and

journal-backed state changes in PL/SQL. That work showed that per-instance tracking and compensation inside the database are feasible, but the design remained without a unified formal model. This work makes temporal allocation semantics explicit on the database layer, defines procedure-inverse compensation, and demonstrates how a process engine can invoke the resource core without re-implementing its semantics.

3 Formal Foundations

This section answers a central design question: how can typed escrow counters and a per-instance allocation journal compose without double-counting pool capacity or losing per-instance exclusivity? We define the core entities, their relationships, temporal constraints, and the semantics of lock-free capacity holds on typed counters. The notation introduced here provides the foundation for the system design in Section 4.

The section proceeds from static entities through time and context, then requests, journal, and constraints, and finally escrow semantics and compensation. Throughout, we refer to a passenger booking resource instance 14A on flight LH710 within allocation context γ . A *binding* is an active allocation of resource instance r in allocation context γ .

3.1 Notation

Table 11 (Appendix: Mathematical Notation) summarises the mathematical notation used throughout this section. Readers will encounter allocation context γ , allocation journal $\mathcal{J}$, active binding registry $\mathcal{L}$, and journal-derived active count $N_{\mathcal{J}}$ repeatedly. Allocation context γ names the temporal frame of a booking. Allocation journal $\mathcal{J}$ records lifecycle transitions. Active binding registry $\mathcal{L}$ and count $N_{\mathcal{J}}$ derive who currently holds which resource instances and how many pool slots are consumed.

Allocation context γ names the scheduled flight interval for LH710. A **reserved** row in allocation journal $\mathcal{J}$ records that recipient u holds resource instance 14A. While the binding is active, pair $(\gamma, 14A)$ belongs to active binding registry $\mathcal{L}$, which enforces exclusivity. Journal-derived active count $N_{\mathcal{J}}(\gamma, \text{Economy})$ records how many Economy seats currently hold an active binding in that flight.

3.2 Resource Model

Heterogeneous resource management requires both interchangeable types and individually tracked instances. For flight LH710, resource types are cabin classes such as **Economy**. Resource instances are numbered seats such as 14A. The model therefore distinguishes resource types from resource instances and the functions that relate them.

Resource Types and Hierarchy

Resources of the same type are functionally interchangeable (e.g. seat 25A and 32A, both window seats in Economy class), yet each resource instance r maintains its unique

identity and is tracked individually. Interchangeability means substitutability: any resource instance of resource type τ can fulfill a request for that type. It does not mean a loss of individual identity or tracking.

Definition 3.1 (Resource Type and Hierarchy). Let $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_m\}$ be a finite set of *resource types*. Each resource type $\tau \in \mathcal{T}$ represents a class of interchangeable resources with identical functional characteristics.

A *type hierarchy* is encoded by a function $\eta: \mathcal{T} \rightarrow \mathbb{N}$ that assigns a *hierarchy level* to each resource type. For practical purposes, we often assume a total ordering $\preceq$ on resource types such that:

$$\tau_i \prec \tau_j \iff \eta(\tau_i) < \eta(\tau_j) \quad (1)$$

In practical terms, resource types correspond to service classes such as First Class, Business Class or Economy. The hierarchy supports flexible substitution when a requested type is unavailable.

Example 3.1. For an airline cabin configuration: $\eta(\text{First Class}) = 1$, $\eta(\text{Business}) = 2$, $\eta(\text{Premium Economy}) = 3$, $\eta(\text{Economy}) = 4$.

Thus, First Class $\prec$ Business $\prec$ Premium Economy $\prec$ Economy.

Type Substitution

When the requested resource type is unavailable, the system may offer alternative types based on the hierarchy. The substitution relation below captures which alternatives a deployment allows.

Definition 3.2 (Substitution Relation). A *substitution relation* $\mathcal{S} \subseteq \mathcal{T} \times \mathcal{T}$ is a binary relation where $(\tau_i, \tau_j) \in \mathcal{S}$ indicates that a resource instance of resource type τ_j may be offered as a substitute when the requested resource type τ_i is unavailable.

Not all type pairs allow for substitution. The relation is governed by business rules and is generally not symmetric:

Definition 3.3 (Substitution Type). For substitute pair $(\tau_i, \tau_j) \in \mathcal{S}$, the *substitution type* is determined by hierarchy level η :

$$\text{subtype}(\tau_i, \tau_j) = \begin{cases} \text{upgrade} & \text{if } \eta(\tau_j) < \eta(\tau_i) \\ \text{downgrade} & \text{if } \eta(\tau_j) > \eta(\tau_i) \\ \text{lateral} & \text{if } \eta(\tau_j) = \eta(\tau_i) \wedge \tau_i \neq \tau_j \end{cases} \quad (2)$$

If resource type **Economy** is full on LH710, the system may offer **Premium Economy** when $(\text{Economy}, \text{Premium Economy}) \in \mathcal{S}$. Substitution type *subtype* then classifies the offer as an upgrade.

Substitution Cost. Each substitution carries an associated cost adjustment:

Definition 3.4 (Cost Adjustment Function). The *cost adjustment function* $c: \mathcal{S} \rightarrow \mathbb{R}$ maps each allowed substitution to an abstract adjustment factor. This factor can represent any domain-specific trade-off:

- cost adjustment $c(\tau_i, \tau_j) > 0$: associates a higher cost with substitute resource type τ_j than with requested resource type τ_i
- $c(\tau_i, \tau_j) < 0$: associates a lower cost with substitute resource type τ_j than with requested resource type τ_i
- $c(\tau_i, \tau_j) = 0$: no cost adjustment (equivalent cost)

Resource Assets. Contained pools such as airline seats require a physical or logical container that bounds capacity. Shared resources such as meeting attendees may omit that container and use $a = \perp$ instead.

Definition 3.5 (Resource Asset). Let $\mathcal{A} = \{a_1, a_2, \dots, a_p\}$ be a finite set of *resource assets*. Each resource asset $a \in \mathcal{A}$ is a physical container (e.g. an aircraft) that holds multiple resource instances and may be named by allocation contexts for contained pools. The same asset cannot simultaneously serve two operational contexts whose scheduled intervals overlap.

Resource Instances

Definition 3.6 (Resource Instance). Let $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$ be a finite set of *resource instances*. Each resource instance $r \in \mathcal{R}$ is a uniquely identifiable physical entity. It may not participate in two conflicting active bindings at once.

Definition 3.7 (Recipient Set). Let $\mathcal{U}$ be the set of *recipients* (users or other assignees) to whom resource instances may be allocated within an allocation context.

Resource instance 14A has $\text{type}(14A) = \text{Economy}$ and $\text{asset}(14A) = \text{aircraft}$ bound to the LH710 flight.

The relationship between resource instances, resource types and resource assets is given by the following functions:

Definition 3.8 (Instance Classification Functions). The function $type: \mathcal{R} \rightarrow \mathcal{T}$ maps each resource instance to its corresponding resource type. The function $asset: \mathcal{R} \rightarrow \mathcal{A} \cup \{\perp\}$ maps each resource instance to its containing resource asset, or to the null value $\perp$ when the instance is not bound to any asset.

For a given resource type τ , we denote the set of all resource instances of that type as:

$$\mathcal{R}_\tau = \{r \in \mathcal{R} \mid type(r) = \tau\} \quad (3)$$

Given functions $type$ and $asset$, each resource asset $a \in \mathcal{A}$ induces a capacity for each resource type $\tau \in \mathcal{T}$:

Definition 3.9 (Capacity Function). The *capacity function* $K: \mathcal{A} \times \mathcal{T} \rightarrow \mathbb{N}_0$ specifies the total number of instances of resource type τ available on resource asset a :

$$K(a, \tau) = |\{r \in \mathcal{R} \mid asset(r) = a \wedge type(r) = \tau\}| \quad (4)$$

Remark. Capacity function $K(a, \tau)$ counts resource instances bound to resource asset a with resource type $type(r) = \tau$. It applies when allocation context γ names a resource asset a . For shared allocation ($asset(r) = \perp$), availability follows interval overlap instead (Section 3.3 and the exclusivity constraints below). Instances marked *intrinsically unavailable* remain in $K(a, \tau)$ but are excluded at *instance selection* by the Intrinsic Admission Constraint (Section 3.5), not by reducing the declared pool size.

Physical Admission and Lifecycle Bindings

Temporal allocation within contexts (Section 3.3) records *who* holds *which* resource instance in *which* event and interval. Admission and lifecycle are separate concerns. Whether a resource instance can be booked at all is distinct from how a particular booking progresses once opened. The definitions below capture that split. They do not introduce a second source of allocation truth.

Definition 3.10 (Intrinsic State). The *intrinsic state* captures the current physical operability of a resource instance independent of any particular booking. Formally, the intrinsic state function $\sigma_{\text{inst}}: \mathcal{R} \rightarrow \Sigma_{\text{inst}}$ assigns the *current* intrinsic state to each resource instance $r \in \mathcal{R}$. The model does not index intrinsic state by time.

Remark. In our implementation, the intrinsic state set is $\Sigma_{\text{inst}} = \{\text{available}, \text{unavailable}\}$. Each deployment may configure a finite set, but the model treats *unavailable* as physical unfitness without a fixed end date (retirement, long-term withdrawal from service), not as a substitute for interval-scheduled maintenance.

Definition 3.11 (Allocation State). The *allocation state* captures the lifecycle of a single *binding* between a recipient, a resource instance and an allocation context. It is recorded as entries in the allocation journal, not stored on the resource instance itself. Each deployment configures a finite set Σ_{alloc} of journal-recorded states (e.g. **reserved**, **confirmed**, **cancelled**). The partition into active and release states, and the permitted transitions between them, are defined by the allocation state machine.

Allocation state governs bindings within allocation context γ and is bound to scheduled interval I_γ . Intrinsic state function σ_{inst} governs whether a *new* binding may be opened at write time, not whether an existing confirmed binding remains valid. A resource instance with intrinsic state $\sigma_{\text{inst}}(r) = \text{unavailable}$ blocks new admissions at entry time t_j but does not erase existing journal history. Interval-scheduled maintenance uses a maintenance allocation context and journalled state **blocked** on overlapping operational contexts (Section 3.5). Intrinsic state function σ_{inst} governs *physical admission*. Allocation states in set Σ_{alloc} govern the *booking lifecycle*.

3.3 Temporal Model

Time is fundamental to resource planning. For LH710, allocation context γ is bound to one aircraft and a scheduled departure interval. We formalise temporal aspects through time points, intervals and allocation contexts.

Time Domain

Definition 3.12 (Time Domain and Intervals). Let $\mathbb{T} = \{t \in \mathbb{R} \mid t \geq 0\}$ be the continuous *time domain*. In practice, we discretise to a suitable granularity (e.g. seconds, minutes).

A *time interval* is a closed subset $I = [t_s, t_e] \subseteq \mathbb{T}$ of the time domain, where $t_s \leq t_e$. We denote:

- $\text{start}(I) = t_s$ as the start time
- $\text{end}(I) = t_e$ as the end time
- $|I| = t_e - t_s$ as the duration

Two time intervals I_1 and I_2 *overlap* if their intersection is non-empty:

$$\text{overlaps}(I_1, I_2) \iff I_1 \cap I_2 \neq \emptyset \quad (5)$$

This predicate captures exactly the interval relations that are neither strictly *before* nor strictly *after* in Allen’s interval algebra (Allen, 1983). The exclusivity constraints in Section 3.5 rely on it to reject conflicting bindings across contexts.

The scheduled intervals of flights LH710 and LH720 may overlap. The same rule applies when a worker is invited to two meetings whose intervals intersect.

Allocation Context

Resources are allocated within temporal frames called *contexts*. The running example flight LH710 is one such context γ .

Definition 3.13 (Allocation Context). An *allocation context* is a tuple $\gamma = (a, \iota, I_\gamma)$ where:

- $a \in \mathcal{A} \cup \{\perp\}$ is an optional bound *resource asset* (e.g. an aircraft, or the null value $\perp$ for shared-resource contexts)
- ι is a *context label* used to denote the allocation context in domain terms (e.g. flight designator LH710). Allocation contexts $\gamma, \gamma' \in \Gamma$ are distinct even when labels coincide, and are distinguished by resource asset a and scheduled time interval I_γ of the context.
- $I_\gamma = [t_s^\gamma, t_e^\gamma]$ is the *scheduled time interval* of the allocation context

We denote the set of all allocation contexts as Γ .

Remark (Asset Binding). When allocation context γ names resource asset a , contained resource instances on that asset are available in γ . When $a = \perp$, resource instances from any asset may be allocated to γ based on scheduled interval I_γ alone.

Temporal Availability

The availability of resources depends on both time and allocation context. Availability at pool level under concurrent load is formalised in Section 3.6.

Timeout Mechanism

Reservations are time-bound to provide a configurable hold window during which a workflow may confirm or cancel. If the workflow completes in time, the allocation progresses. Otherwise the reservation is released and the resource instance becomes available again. Expiry time t_{expire} corresponds to the hold window analysed in Section 3.6.

Definition 3.14 (Reservation Timeout). A reservation created at time point t_{created} with timeout duration Δ_{timeout} expires at:

$$t_{\text{expire}} = t_{\text{created}} + \Delta_{\text{timeout}} \quad (6)$$

If not confirmed by expiry time t_{expire} , the reservation transitions to a release allocation state (e.g. cancelled).

3.4 Request Model and Lifecycle

Resource Requests

Set $\mathcal{Q}$ is the abstract request language. Requests by category and requests for a named resource instance differ only in how a target instance is chosen. Validation, journal writes, and escrow admission follow the same rules once a concrete resource instance is fixed.

An instance-based request for LH710 might fix resource instance $r_q = 14A$ with quantity $k_q = 1$. A request by category might ask for $k_q = 2$ seats of resource type **Economy** in allocation context γ_q without naming instances.

Definition 3.15 (Resource Request). A *resource request* is a tuple $q = (\gamma_q, \tau_q, r_q, u_q, k_q, t_q)$ where:

- $\gamma_q \in \Gamma$ is the allocation context
- $u_q \in \mathcal{U}$ is the recipient
- $k_q \in \mathbb{N}$ is the requested quantity
- $t_q \in \mathbb{T}$ is the request time
- $\tau_q \in \mathcal{T} \cup \{\perp\}$ is the requested resource type (or $\perp$ when unused)
- $r_q \in \mathcal{R} \cup \{\perp\}$ is the target resource instance (or $\perp$ when unused)

Resource request q is *valid* if it satisfies exactly one of:

- **By category:** $r_q = \perp$, requested resource type $\tau_q \in \mathcal{T}$, and quantity $k_q \geq 1$. The system selects up to k_q instances of type τ_q in allocation context γ_q .
- **By instance:** $r_q \in \mathcal{R}$, $\tau_q = \text{type}(r_q)$, and $k_q = 1$. The request targets resource instance r_q .

We denote the set of all valid resource requests by $\mathcal{Q}$.

Allocation Lifecycle

The allocation lifecycle is modelled as a finite state machine over journalled allocation states. Each state in Σ_{alloc} is either *active* or *release*; capacity and overlap rules below consult Σ_{active} only.

Definition 3.16 (Lifecycle Events). Let $\mathcal{E}$ be a finite set of *lifecycle events* configured per deployment (e.g. confirm, cancel, timeout). Each event names an operation that may append a new journal entry.

Definition 3.17 (Allocation State Machine). The *allocation state machine* records the lifecycle of an allocated resource instance in the journal.

$$M = (\Sigma_{\text{alloc}}, \sigma_0, \Sigma_{\text{active}}, \Sigma_{\text{release}}, \delta)$$

where:

- Σ_{alloc} is a finite set of *allocation states* that may appear in journal entries, configurable per deployment (e.g. **reserved**, **confirmed**, **cancelled**).
- σ_0 denotes the implicit pre-binding state before any journal entry exists for a binding; it is not an element of Σ_{alloc} and is not written to the journal.
- $\Sigma_{\text{active}}, \Sigma_{\text{release}} \subseteq \Sigma_{\text{alloc}}$ with $\Sigma_{\text{active}} \cap \Sigma_{\text{release}} = \emptyset$. States in Σ_{active} withhold the binding; states in Σ_{release} end it.
- $\delta \subseteq \Sigma_{\text{alloc}} \times \mathcal{E} \times \Sigma_{\text{alloc}}$ is a partial *transition relation* specifying which allocation-state changes each lifecycle event permits.

Each deployment configures δ together with Σ_{active} and Σ_{release} . A reference configuration for the airline PoC appears in Section 4.

Allocation Journal

For each pair (γ, r) , binding status is fixed by the latest journal entry for that pair. Allocation journal $\mathcal{J}$ is the append-only record of lifecycle transitions on those bindings.

Definition 3.18 (Allocation Journal). The *allocation journal* records every lifecycle transition of an allocation binding. Entries are appended in time order and are not changed once recorded. Each journal entry names recipient, allocation context, resource type, resource instance, allocation state, timestamp, and optional metadata μ_j . Allocation journal $\mathcal{J}$ is a sequence. Each journal entry $j \in \mathcal{J}$ is a tuple

$$j = (id_j, t_j, \gamma_j, \tau_j, u_j, r_j, \sigma_j, \mu_j) \tag{7}$$

where:

- $id_j \in \mathbb{N}$ is a unique *journal entry identifier*
- $t_j \in \mathbb{T}$ is the entry timestamp
- $\gamma_j \in \Gamma$ is the allocation context
- $\tau_j \in \mathcal{T}$ is the resource type
- $u_j \in \mathcal{U} \cup \{\perp\}$ is the recipient, or $\perp$ when no recipient is bound
- $r_j \in \mathcal{R}$ is the resource instance
- $\sigma_j \in \Sigma_{\text{alloc}}$ is the allocation state
- μ_j is optional metadata

Definition 3.19 (Derived Binding Status). For allocation context γ and resource instance r , let j be the journal entry with maximal timestamp t_j among $\{j' \in \mathcal{J} \mid \gamma_{j'} = \gamma \wedge r_{j'} = r\}$. Derived status $current(\gamma, r)$ is

$$current(\gamma, r) = \begin{cases} \text{available} & \text{if no such } j \text{ exists, or } \sigma_j \in \Sigma_{\text{release}} \\ \sigma_j & \text{if } \sigma_j \in \Sigma_{\text{active}} \end{cases} \quad (8)$$

When $current(\gamma, r) \in \Sigma_{\text{active}}$, that binding is in force and conflicting allocations are excluded by the exclusivity constraints below.

Remark. Status **available** is derived from the journal. It is not an element of Σ_{alloc} and matches initial state σ_0 when no active binding holds. Journal entries describe binding lifecycle within a context, not intrinsic resource existence (above). Release states such as **completed** end the binding without retiring the resource instance.

For resource instance 14A on flight LH710, a typical trace runs from implicit availability to allocation state **reserved**, then to **confirmed**. Each step adds one entry to allocation journal $\mathcal{J}$. Physical storage and commit strategy are described in Section 4.

3.5 Constraint System

A valid allocation must satisfy several constraints.

Definition 3.20 (Context Journal Slice). For an allocation context $\gamma \in \Gamma$, the *context journal slice* is

$$\mathcal{J}_\gamma = \{j \in \mathcal{J} \mid \gamma_j = \gamma\}. \quad (9)$$

Capacity constraints below apply to contained allocation contexts. Exclusivity constraints are stated over active binding registry $\mathcal{L}$.

Definition 3.21 (Journal-Derived Active Count). The *journal-derived active count* $N_{\mathcal{J}}(\gamma, \tau)$ is the number of resource instances in allocation context γ whose *current* allocation state is active and whose resource type is τ :

$$N_{\mathcal{J}}(\gamma, \tau) = |\{r \in \mathcal{R} \mid \text{current}(\gamma, r) \in \Sigma_{\text{active}} \wedge \text{type}(r) = \tau\}| \quad (10)$$

Definition 3.22 (Active Binding Registry). Let $\mathcal{L} \subseteq \Gamma \times \mathcal{R}$ be the set of bindings that currently hold an active allocation state. For each binding $(\gamma, r) \in \mathcal{L}$, let $\lambda(\gamma, r) \in \mathbb{N}$ denote the journal entry identifier of the authoritative active entry for that binding. Each binding $(\gamma, r) \in \mathcal{L}$ is backed by exactly one journal entry j with $\lambda(\gamma, r) = id_j$. **Consistency invariant (registry vs. journal).** Membership in active binding registry $\mathcal{L}$ matches journal-derived active status:

$$(\gamma, r) \in \mathcal{L} \iff r \neq \perp \wedge \text{current}(\gamma, r) \in \Sigma_{\text{active}}. \quad (11)$$

Thus active binding registry $\mathcal{L}$ is a materialised membership view of journal-derived state, not an independent allocation rule. As a set, $\mathcal{L}$ contains each binding at most once. Exclusivity within a single allocation context is enforced separately below. Global exclusivity on resource instance r across overlapping contexts is governed solely by the Exclusivity Constraint on $\mathcal{L}$ below.

Remark (Aggregate vs. instance-level views). Journal-derived active count $N_{\mathcal{J}}(\gamma, \tau)$ and active binding registry $\mathcal{L}$ are both derived from status $\text{current}(\gamma, r)$, but they answer different questions. Count $N_{\mathcal{J}}(\gamma, \tau)$ supports the capacity constraint. Registry $\mathcal{L}$ supports per-instance exclusivity. Table 2 summarises the distinction.

Table 2: Journal-derived active count $N_{\mathcal{J}}$ vs. active binding registry $\mathcal{L}$

Construct	Question	Granularity	Used for
$N_{\mathcal{J}}(\gamma, \tau)$	How many?	Per context and type τ	Capacity constraint
$\mathcal{L}$	Which instances?	Per (context γ , instance r) pair	Exclusivity constraint

Definition 3.23 (Capacity Constraint). For all allocation contexts $\gamma \in \Gamma$ with $a_\gamma \neq \perp$ (contained allocation) and all resource types $\tau \in \mathcal{T}$:

$$N_{\mathcal{J}}(\gamma, \tau) \leq K(a_\gamma, \tau) \quad (12)$$

Definition 3.24 (Exclusivity Constraint). A resource instance cannot participate in two *conflicting active bindings* at once. The rule depends on the type of allocation:

- **Contained allocation** (one context bound to an asset): within a single allocation context γ , at most one active binding on resource instance r .

- **Shared allocation** (contexts with overlapping intervals): across any two allocation contexts γ_1, γ_2 whose scheduled time intervals $I_{\gamma_1}, I_{\gamma_2}$ overlap, at most one active binding on resource instance r may exist.

Formally, *within-context exclusivity*: active binding registry $\mathcal{L}$ is a set. At most one active binding per allocation context γ and resource instance r .

Exclusivity across contexts:

$$\forall (\gamma_1, r), (\gamma_2, r) \in \mathcal{L}: \gamma_1 \neq \gamma_2 \implies \neg \text{overlaps}(I_{\gamma_1}, I_{\gamma_2}) \quad (13)$$

Definition 3.25 (Type Constraint). An allocation for resource type τ can only use resource instances r of that type:

$$\forall j \in \mathcal{J}: r_j \neq \perp \implies \text{type}(r_j) = \tau_j \quad (14)$$

Definition 3.26 (Intrinsic Admission Constraint). An append of journal entry j with $r_j \neq \perp$ and $\sigma_j \in \Sigma_{\text{active}}$ is admissible only if $\sigma_{\text{inst}}(r_j) = \text{available}$ at entry time t_j (evaluated immediately before the write).

Remark. The constraint is checked only at admission time. Later changes to intrinsic state function σ_{inst} do not invalidate existing bindings. For scheduled maintenance, withholding uses journalled state **blocked** (Section 3.5), not σ_{inst} .

Maintenance Contexts and Cross-Context Withholding

Interval-scheduled unavailability on contained resources is not modelled by changing intrinsic state function σ_{inst} . Deployments distinguish *operational* allocation contexts (ordinary use, e.g. flights) from *maintenance* contexts (intervals during which specific resource instances must not be newly admitted on overlapping operational schedules). Withholding is expressed through journalled allocation state **blocked** $\in \Sigma_{\text{active}}$: the resource instance remains intrinsically **available**, but $\text{current}(\gamma, r) = \text{blocked}$ rejects conflicting admissions in allocation context γ .

If seat 14A enters maintenance during an interval that overlaps LH710, a **blocked** entry in context γ prevents new admissions on that flight without changing intrinsic state on the resource instance.

For contained allocation, when maintenance and operational contexts share an asset and their intervals overlap, the model aligns withholding across context types:

- **Retroactive propagation.** When a maintenance context is introduced, append **blocked** entries in each overlapping operational context for the affected resource instances.

- **Preemptive propagation.** When an operational context is created, append **blocked** entries in that context for resource instances already under maintenance on overlapping intervals.

Shared allocation does not use this propagation. Exclusivity across overlapping contexts follows from active binding registry $\mathcal{L}$ alone. Section 4 describes the corresponding context-creation procedures.

3.6 Escrow Semantics on Capacity Counters

On contained pools, where allocation context γ names a resource asset, the Capacity Constraint requires that journal-derived count $N_{\mathcal{J}}(\gamma, \tau)$ does not exceed $K(a_{\gamma}, \tau)$. Shared allocation relies on journal overlap and exclusivity on active binding registry $\mathcal{L}$ without typed capacity counters (Section 3.8). Under concurrent load, reservations may be *in progress*. They are buffered but not yet finalised.

Classical escrow transactions (O’Neil, 1986), generalising the earlier hot-field concurrency control of Gawlick and Kinkade (1985), already capture this pattern for a single aggregate counter.

In our implementation, the same semantics are provided by Oracle’s lock-free reservation (Oracle Corporation, 2026b), which was part of the inherited platform rather than introduced by this work.

Section 2.1 describes escrow on one aggregate counter. The same pattern applies to typed counters indexed by allocation context γ and resource type τ , with session-local buffers Δ_w , held capacity H , and admission checks that count other sessions’ in-flight buffers.

The journal-derived active count alone is not sufficient for admission under concurrency. Two sessions can each pass a journal snapshot while both extend escrow buffers. Held capacity H therefore includes committed counter base N_{cnt} and all tentative escrow buffers Δ_w from concurrent transactions.

Capacity quantities.

Definition 3.27 (Available Capacity, Committed Counter Base, and Held Capacity). For contained allocation contexts ($a_{\gamma} \neq \perp$) and resource type τ , the *available capacity* in a committed journal snapshot is:

$$A(\gamma, \tau) = K(a_{\gamma}, \tau) - N_{\mathcal{J}}(\gamma, \tau) \tag{15}$$

The *committed counter base* $N_{\text{cnt}}(\gamma, \tau)$ is the aggregate capacity already committed on the typed counter for allocation context γ and resource type τ . It is the base value of `active_count` before session-local `RESERVABLE` buffers are applied.

Held capacity $H(\gamma, \tau, t)$ is the committed counter base $N_{\text{cnt}}(\gamma, \tau)$ plus all tentative escrow buffers $\Delta_w(\gamma, \tau)$ from concurrent transactions. It is the load the pool must respect when extending a buffer:

$$H(\gamma, \tau, t) = N_{\text{cnt}}(\gamma, \tau) + \sum_{w \in \mathcal{W}_t} \Delta_w(\gamma, \tau) \quad (16)$$

where set $\mathcal{W}_t$ denotes the concurrent transactions $\mathcal{W}$ active at time point t .

Escrow admission.

Definition 3.28 (Reservable Delta Buffer and Escrow Admission Rule). Let $\mathcal{W}$ be the set of concurrent transactions touching the system. While a reservation is in progress, a concurrent transaction may hold a *tentative* capacity adjustment that is not yet committed. This is an escrow *hold* in O’Neil’s sense. Each concurrent transaction $w \in \mathcal{W}$ maintains an integer reservable-delta buffer $\Delta_w(\gamma, \tau)$ for allocation context γ and resource type τ . A reservation contributes $+1$, its inverse -1 . Abort or rollback resets the buffer to 0.

An extension of escrow buffer $\Delta_w(\gamma, \tau)$ by concurrent transaction w is *admissible* only if held capacity $H(\gamma, \tau, t)$ including that extension does not exceed container capacity $K(a_\gamma, \tau)$ on resource asset a_γ :

$$H(\gamma, \tau, t) = N_{\text{cnt}}(\gamma, \tau) + \sum_{w' \in \mathcal{W}_t} \Delta_{w'}(\gamma, \tau) \leq K(a_\gamma, \tau). \quad (17)$$

The `RESERVABLE` layer evaluates this inequality when the session applies each buffer increment, counting other sessions’ in-flight buffers. If the extension would violate the rule, the statement fails immediately and no journal hold is created for that attempt. Merging buffered deltas into the committed counter base still occurs when the main transaction commits after confirm.

Lock freedom is inherited from classical escrow and, for this work, from `RESERVABLE` session buffering. Concurrent transactions hold session-local escrow buffers Δ_w without row-level locks on the counter for allocation context γ and resource type τ , while the escrow layer enforces $H(\gamma, \tau, t) \leq K(a_\gamma, \tau)$ on each buffer extension with visibility of peer buffers.

To answer how many seats are left on a contained pool, we subtract held capacity $H(\gamma, \tau, t)$ from container capacity $K(a_\gamma, \tau)$ on the named resource asset. Held capacity includes the committed counter base and every in-flight escrow buffer from concurrent

sessions. Available capacity $A(\gamma, \tau) = K(a_\gamma, \tau) - N_{\mathcal{J}}(\gamma, \tau)$ counts only bindings already visible in the journal and omits open holds. The counter base $N_{\text{cnt}}(\gamma, \tau)$ alone can overstate free seats until peer sessions commit. Reporting views that need audit-complete bindings may use the active count $N_{\mathcal{J}}(\gamma, \tau)$ derived by the journal instead. The two notions agree after every main transaction commits or is fully compensated (Section 3.6).

Remark (Why both layers). Escrow on typed counters is *necessary but not sufficient* for heterogeneous resource management. It keeps aggregate capacity within bounds under concurrency but cannot express per-instance exclusivity, the allocation state machine, or the inverse procedures required by sagas. Those concerns are addressed by allocation journal $\mathcal{J}$ and the compensation calculus below, layered *on top of* the inherited escrow primitive. The two artifacts cooperate as formalisations of different aspects of the same reservation act, rather than as alternatives.

Coherence of Journal, Counter, and Escrow. Allocation journal $\mathcal{J}$ and active binding registry $\mathcal{L}$ record who holds which resource instance, support exclusivity, and preserve the audit trail. Typed capacity counters with escrow buffers Δ_w govern how many units of resource type τ may still be admitted in allocation context γ under concurrent writers. Neither mechanism replaces the other. They must be read together during a reservation hold.

Reservation hold. While a reservation hold is open on a contained pool, aggregate admission is governed by held capacity H , not by journal-derived active count alone. Instance exclusivity and the audit trail follow allocation journal $\mathcal{J}$ and active binding registry $\mathcal{L}$. The same reservation must not consume one pool slot in journal-derived active count $N_{\mathcal{J}}$ and again in escrow buffer Δ_w . Transition **reserved** $\rightarrow$ **confirmed** does not change aggregate pool consumption. After commit or full compensation, committed counter base $N_{\text{cnt}}(\gamma, \tau)$ matches journal-derived active count $N_{\mathcal{J}}(\gamma, \tau)$ for pool-consuming contained allocations. Phase order and artifact updates appear in Section 4.7 (Table 3 (Section 4.7)).

Shared allocation does not use typed counters or escrow buffers Δ_w . Exclusivity there follows allocation-journal overlap only (Section 3.8).

3.7 Compensation Model

Allocation journal $\mathcal{J}$ is append-only. Compensation therefore appends inverse entries rather than deleting forward rows. To ensure consistency in the presence of failures, we employ a compensation-based transaction model inspired by the Saga pattern (Garcia-Molina and Salem, 1987). The procedure-inverse pairing formalised below is an instance of the compensation dependencies made precise in extended-transaction frameworks such

as ACTA (Chrysanthis and Ramamritham, 1990), where a committed step is undone by a semantically inverse step rather than by physical rollback.

Procedures and Inverses.

Definition 3.29 (Domain Procedure). Let $\mathcal{P} = \{p_1, p_2, \dots, p_k\}$ be the set of *domain procedures* that modify the allocation state. Each domain procedure $p \in \mathcal{P}$ is a function:

$$p: \mathcal{J} \times \mathbb{T} \rightarrow \mathcal{J}' \quad (18)$$

where updated journal state $\mathcal{J}'$ represents allocation journal $\mathcal{J}$ after domain procedure p executes.

Definition 3.30 (Compensating Procedure). For each forward domain procedure $p \in \mathcal{P}$, a *compensating procedure* p^{-1} undoes the *effect* of p on the allocation state while *preserving* the audit trail. Allocation journal $\mathcal{J}$ retains both the forward entry and the compensating entry. Formally, compensating procedure $p^{-1} \in \mathcal{P}$ satisfies semantic equivalence with the logical state before p ran:

$$p^{-1}(p(\mathcal{J}, t), t') \equiv_{\text{semantic}} \mathcal{J} \quad (19)$$

Saga Pattern Formalisation.

Definition 3.31 (Allocation Saga). An *allocation saga* is an ordered sequence of domain procedures $S = \langle p_1, p_2, \dots, p_n \rangle$ executed as one logical transaction. If domain procedure p_i fails after $p_1, \dots, p_{i-1}$ have already committed, the system runs the compensating procedures in reverse order:

$$\text{compensate}(S, i) = \langle p_{i-1}^{-1}, p_{i-2}^{-1}, \dots, p_1^{-1} \rangle \quad (20)$$

Example 3.2 (Flight-wide cancellation). Flight LH710 has active bindings on resource instances $r_1, \dots, r_k$ in allocation context γ . Releasing them all is saga $S = \langle p_1, \dots, p_k \rangle$, where each domain procedure p_j cancels the binding on r_j . If p_i fails after $p_1, \dots, p_{i-1}$ have already committed, $\text{compensate}(S, i)$ runs $p_{i-1}^{-1}, \dots, p_1^{-1}$ in order. Journal rows from the forward cancellations and from the compensating steps all remain in allocation journal $\mathcal{J}$. Orchestration and failure handling for this pattern appear in Section 4.8.

3.8 Limitations of the Model

This formal model makes several simplifying assumptions:

1. **Deterministic capacity:** capacity function K is fixed per allocation context γ and does not vary with time within that context. Dynamic pool changes are not modelled.
2. **Minimal constraints:** Only capacity, exclusivity, type and intrinsic-admission invariants are formalised. The model does not include a general constraint or preference language. A richer constraint layer is discussed in Section 6.2.
3. **Asymmetric allocation modes:** Escrow semantics apply to contained pools where allocation context γ names a resource asset. Shared allocation relies on journal overlap and exclusivity without typed capacity counters.
4. **Independent allocations:** Conflict detection across contexts does not extend to atomic multi-context bookings (e.g. connecting flights). A grouping abstraction would be required.
5. **Admission-time intrinsic state:** Physical operability is modelled as intrinsic state function $\sigma_{\text{inst}}(r)$, evaluated when a new binding is admitted. The formal layer does not maintain an intrinsic timeline indexed by time. Interval maintenance is represented through maintenance allocation contexts and journal state **blocked** (Section 3.5).

Section 7.2 and Section 6 discuss these boundaries further.

4 System Design

This section maps the formal model from Section 3 to an applicable schema, PL/SQL packages, and the procedure surface. It first treats typed escrow counters and the allocation journal, then data structures, the hold transaction, compensation, and manual validation. Further implementation detail, schema mapping, and validation scenarios, appear in Appendix: Implementation Reference.

4.1 Escrow and Journal as Cooperating Layers

Section 3 introduced typed escrow counters for pool admission and the allocation journal for per-instance lifecycle and compensation. They remain separate tables and packages in the implementation and are reconciled at commit (Section 4.7). The division of labour is concrete. The counters bound aggregate admission under concurrency without taking row locks. The journal carries everything tied to a specific instance, namely its binding, its lifecycle state, and the inverse rows needed to reverse a failed step while keeping the audit history intact.

4.2 System Architecture

Resource semantics live in the database: business logic in PL/SQL on **Oracle Database 26ai**, with Oracle APEX Workflow (Oracle Corporation, 2026a) at the process boundary (Section 5). Resource types, temporal contexts, and lifecycle bindings are modelled in schema and packages, with every transition recorded in an append-only journal. Workflows call a small procedure surface instead of scattering admission, overlap checks, and compensation across process scripts.

Figure 1 shows the end-to-end architecture. Resource types, contexts, state rules, and substitution are configured in the resource core. Workflow steps declare lifecycle intent and reach packaged procedures through the *Resource Runtime API*.

The runtime path is thin at the workflow boundary. The process declares intent (*reserve*, *confirm*, *cancel*, *complete*). The resource package checks capacity, writes the journal, applies substitution when needed (Section 4.9), and enforces transaction and compensation rules. The asynchronous hold path publishes correlated **CONFIRM** or **CANCEL** events to Transactional Event Queues (TxEventQ) (Oracle Corporation, 2026c). The same session that opened the hold waits and then resolves it (Section 4.7). Dashed elements in Figure 1 denote a planned notification subscriber (Section 6.2), not part of the current implementation.

Workflow plugins call a single PL/SQL API implemented as two separate packages. Only the domain package is visible to workflow authors. The data package persists catalog entities, capacity counters, and journal rows. Journal writes inside the hold window use

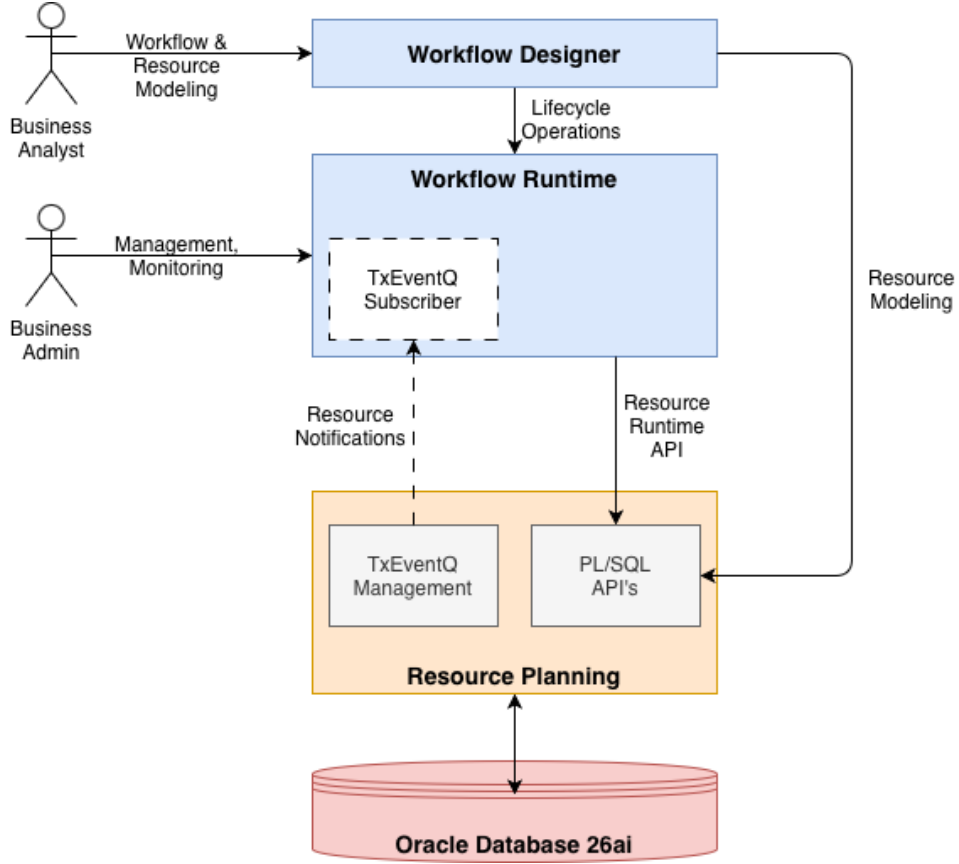


Figure 1: System architecture of the Resource Planning System. Dashed elements denote the planned notification subscription path, not yet implemented.

autonomous transactions so the instance is held exclusively while the main transaction remains open (Section 4.7).

4.3 Domain Language

Process designers interact with the resource core through a small vocabulary of lifecycle actions.

Lifecycle vocabulary is what business analysts and workflow designers must know: actions such as *request*, *confirm*, *cancel* and *complete*. An additional **Extension surface** matters only when adapting the resource package to a new domain: autonomous journal writes, RESERVABLE admission, and procedure-inverse pairs.

A *Request* opens a hold on an instance or categorical quantity and denotes the allocation context, the instance, and the recipient. When the requested resources are unavailable, the *confirm* message may also record the customer’s choice among substitutional counter offers (Section 4.9). Type-based and instance-based requests differ only in how a target

instance is chosen. The domain package dispatches to the same validation, journal, and escrow path once the instance is fixed.

Configurable vs. invariant. Each deployment may configure allocation states, lifecycle events, timeout policy, and the type hierarchy. The core remains invariant: escrow admission on typed counters, procedure-inverse compensation, and an append-only journal. The reservation hold pairs in Table 4 (Section 4.8) are the validation focus. Callers do not invoke inverses directly, because failure and timeout paths trigger them inside the package.

Section 5 shows how APEX Workflow invokes this vocabulary at the process boundary.

4.4 Data Model Design

The relational schema in Oracle Database 26ai instantiates the formal entities from Section 3 in three steps: what resources exist, when they may be bound, and how concurrent bindings are enforced at runtime. See Figure 2:

Table 12 (Appendix: Schema-to-Formal-Model Mapping) maps each formal symbol to its implementation table and the invariant it enforces.

Resource catalog. The catalog describes allocatable entities in three layers:

- **ResourceCategory** represents the resource type $\mathcal{T}$, tagged either *contained* or *shared*.
- **ResourceAsset** represents an asset $\mathcal{A}$: a physical or logical container such as an aircraft. Assets are optional and used where instances belong to a bounded pool.
- **ResourceInstance** represents an individual resource $r \in \mathcal{R}$. Each instance belongs to one category and optionally to one asset, with intrinsic state $\sigma_{\text{inst}}(r)$ on **ResourceInstance.status** reflecting physical operability at admission time.

For contained pools, **AssetCapacity** declares how many units of each category an asset provides. Shared categories do not use capacity counters; availability is derived from interval overlap across contexts.

Allocation contexts. **AllocationContext** implements the context set Γ : a scheduled time interval during which resources may be bound. Each row has a **context_type** discriminator (*default* for operational use, *maintenance* for asset-bound maintenance periods). Contained categories bind their contexts to a specific asset; shared categories use interval-only contexts (e.g. meetings).

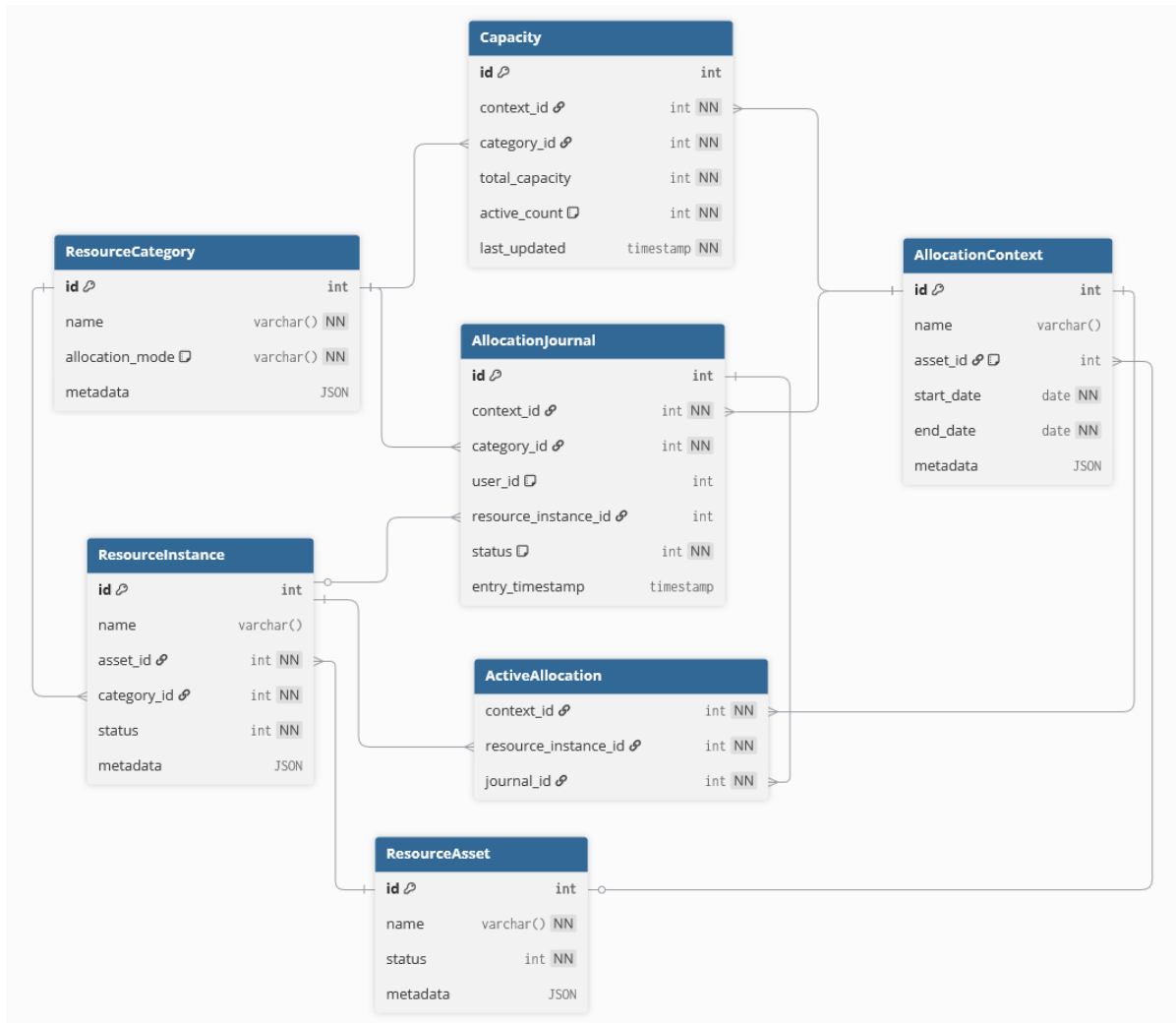


Figure 2: Core database schema for the Resource Planning System

Contained and shared modes. Both modes share the same journal, state machine, and compensation framework. They differ in how a request is admitted:

- **Resource selection:** system-assigned from a pool or specified instance vs. only specified instance
- **Availability scope:** within one context vs. across overlapping intervals
- **Capacity tracking:** typed counters vs. temporal overlap queries

Contained allocation is a specialization where asset-bound pools make cross-context checking unnecessary. As long as the bound asset is available, all contained resources are guaranteed. Comprehensive blocking (Section 4.5) propagates **blocked** journal entries for contained resources. Shared allocation relies on cross-context overlap checks instead (Section 3.5).

Active binding enforcement. Journal-derived state alone is not enough under concurrency: two sessions can both believe the same seat is free before either writes. The implementation therefore materialises the formal active binding set $\mathcal{L}$ as the **ActiveAllocation** table, with a unique constraint on (context_id, resource_instance_id). The row is inserted in the *same autonomous transaction* as the journal entry, so a conflicting second writer fails immediately with a uniqueness violation.

Each row records one active (context γ , instance r) pair. The `journal_id` column references the journal entry that currently authorizes the binding. Complete history remains in **AllocationJournal**. Within each autonomous call, an **INSERT** creates the row for **reserved** or **blocked**, a forward transition **UPDATES** `journal_id`, and a release state **DELETES** the row.

Physical Admission and Lifecycle Bindings

Section 3 separates physical operability from journalled lifecycle state. The implementation keeps each dimension in a separate store and checks each during admission. The same seat can already be bound to a reservation on one flight but remains available for a different flight.

- **Intrinsic state** (σ_{inst}) is stored on **ResourceInstance.status**. $\Sigma_{\text{inst}} = \{\text{available}, \text{unavailable}\}$: **available** permits new bindings; **unavailable** signals open-ended physical unfitness (e.g. retirement or long-term withdrawal from service). Appending a journal entry does not change the intrinsic state. On every path that appends an active journal row with a concrete instance, the domain package reads **ResourceInstance.status** and rejects the write unless it is **available** (admission-time check at t_j).

- **Allocation state** (σ_{alloc}) is recorded only in the allocation journal as part of an affiliation between the allocation context, the instance, and the recipient. The set of allocation states Σ_{alloc} , and the admissible transitions δ are domain-configurable. **Blocked** journal entries (e.g. on short notice or via maintenance propagation) withhold the instance in a given context without changing the intrinsic state.

Admission and conflict checks use both dimensions where applicable. Intrinsic **unavailable** rules out new bindings at admission even when the journal still shows a confirmed allocation in another allocation context, and an active journal entry can block a new request even when the instance is intrinsically **available**.

Scheduling a maintenance allocation context appends **blocked** journal entries on overlapping flights while a **confirmed** booking on a non-overlapping flight remains valid. Alternatively, marking an instance **unavailable** on retirement prevents new admissions without deleting journal history.

4.5 Journal-Based State Management

The **AllocationJournal** table implements the formal journal $\mathcal{J}$ (Section 3.4). Each row records a single lifecycle transition. Previous rows are never modified. Current allocation state is derived from the latest entry per (context γ , instance r) pair, with implicit **available** when no active binding exists.

Transition Validation

Each transition is checked against the state machine M . The implementation requires that the requested event is admissible from the derived current state, that domain-specific preconditions hold (e.g. an unexpired reservation or a context that has not departed), and that journal writes and capacity updates follow the single-transaction hold pattern. During that window the journal binding commits autonomously to prevent double-booking while pool capacity stays in the session-local escrow buffer until confirm or rollback.

Figure 3 shows the airline reference configuration. Solid arrows are forward transitions; dashed red arrows lead to **cancelled**; **blocked** (dashed border) is administrative withholding reached by maintenance propagation and related procedures (Section 4.5), not by the passenger lifecycle arrows shown. Green nodes release the resource, making it available again for allocation. Corrective steps back to an earlier active state (e.g. cancel boarding) appear in Table 4 (Section 4.8).

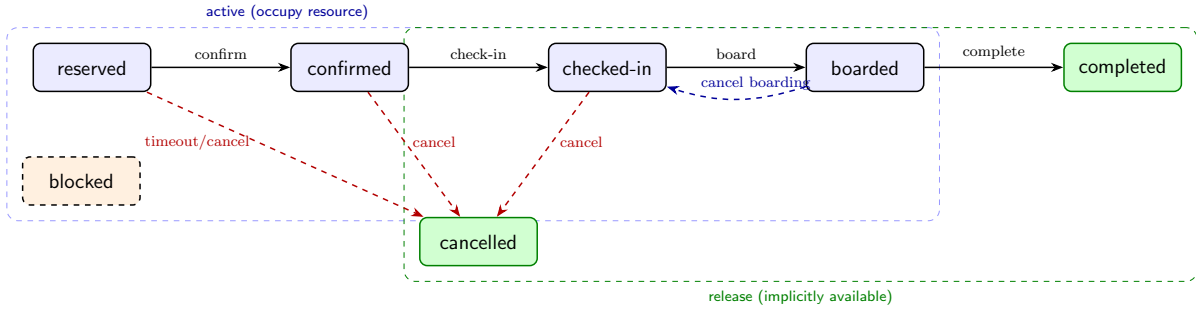


Figure 3: Allocation state machine (airline configuration).

Comprehensive Blocking Across Typed Contexts

Section 3.5 formalises maintenance contexts, a withholding **blocked** state, and retroactive versus preemptive propagation across overlapping intervals. Context-creation procedures apply those rules when allocation contexts are added.

Remark. This propagation mechanism is implemented for contained resources. Shared resources remain governed by cross-context overlap checks: if an attendee is already allocated in an overlapping interval, the meeting allocation request is rejected.

Path A (Retroactive: new maintenance context).

- **Trigger:** A maintenance context is created for contained instances (full asset or subset).
- **Scan:** The procedure finds overlapping operational contexts that already exist.
- **Effect:** It appends a **blocked** journal entry in each affected operational context, regardless of prior journal state.

Path B (Preemptive: new operational context).

- **Trigger:** An operational context is created.
- **Scan:** The procedure finds overlapping maintenance contexts on the same asset or instances.
- **Effect:** It appends **blocked** into the new operational context during creation, in the same transaction as context setup, so instances already under maintenance are unavailable before any allocation is requested.

4.6 Capacity Management

Efficient capacity tracking is essential for reservation systems. Aggregate availability is checked without scanning the journal on each admission. For contained allocation, the design uses reservable capacity counters. For shared allocation, it uses time-overlap queries.

Counter-Based Availability. To track **Capacity**, we store total pool size and current consumption in two columns:

- **total_capacity**: total pool size for the context, copied from **AssetCapacity** at creation
- **active_count**: current consumption in active allocation states (the **RESERVABLE** column)

At context creation, the **total_capacity** is derived from the defined capacity of the bound asset, for each category. Only the **active_count** is incremented on admission and decremented on release. Decrementing **total_capacity** on each allocation would treat remaining seats as the only stored quantity and would lose a stable reference for reconciliation against the asset configuration.

Transitions between active allocation states do not change the escrow counter. For example, **reserved** → **confirmed** leaves **active_count** unchanged. A release to **cancelled** after the counter has been committed decrements **active_count**. A manual cancel or timeout *during* the hold window rolls back the *uncommitted* escrow buffer without having to compensate the counter.

For contained pools, reported availability is **total_capacity** minus **active_count** on the **Capacity** table. That snapshot uses the committed counter base only. It omits in-flight buffers from other sessions and may diverge from journal-derived bindings while holds remain open (Table 3 (Section 4.7)). That is deliberate for simple reporting. Admission does not rely on this snapshot. Each attempt extends a session-local escrow buffer and is checked against held capacity, including buffers from concurrent sessions (Section 4.6). For operational “seats left” under load, use held capacity (Section 3.6) or a journal-derived count. A failed booking after an optimistic display check is expected, not a consistency error.

If detailed counts per allocation state are required (e.g. for reporting), they can be derived from the journal via a separate procedure or view without affecting the core capacity enforcement logic.

Lock-Free Counter Updates. The `active_count` column uses Oracle’s `RESERVABLE` column (Oracle Corporation, 2026b). Each buffer extension is evaluated against `total_capacity` with visibility of concurrent session buffers. If the extension is rejected, no journal row is written. On confirm the buffer merges into the committed counter base. On rollback or timeout it is discarded without a compensating decrement.

Reconciliation. Compensation and in-procedure rollback keep journal rows and `active_count` aligned on the normal hold path. Gaps can still appear when compensation itself fails, the hold ends without a clean cancel or confirm, or counters drift after manual correction. Reconciliation is therefore a separate administrative path and not part of admission.

Each journal write records the parent transaction identifier in the journal metadata. A scheduled job compares journal-derived bindings and consumption against `active_count` and scans for orphaned or stale rows. Examples include holds left in `reserved` beyond the timeout window and active bindings on a context that is already completed. Where drift is found, the procedure rebuilds aggregate consumption from current journal entries for the affected context and category. It can also append compensating entries for rows flagged as inconsistent.

Running reconciliation on a schedule during low-activity periods was chosen to keep repair logic manageable within the scope of this thesis.

4.7 Transaction and Commit Strategy

The reservation lifecycle must preserve a complete audit trail while the client decides, and must hold the resource exclusively for the duration of that decision. We keep one main database transaction open for the full reservation hold window (Figure 4). An autonomous journal commit at reservation time prevents double booking and records the binding before the main transaction finishes. Pool capacity stays in the session-local escrow buffer until confirm or rollback.

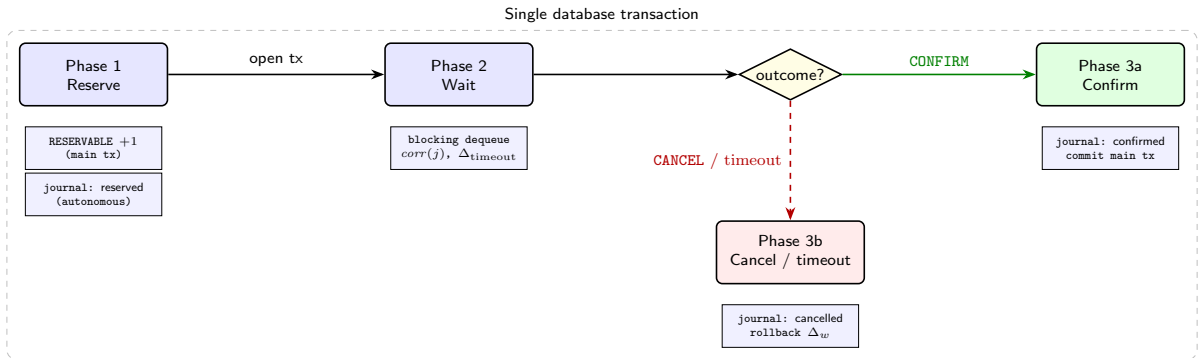


Figure 4: Transaction strategy for the reservation hold window.

Single-Transaction Hold. While the main transaction remains open, the requesting session must already hold the resource. The autonomous journal write at reservation time provides that guarantee: the **ActiveAllocation** row and the journal entry commit immediately, so no concurrent session can claim the same instance. For resource pools, a session-local escrow buffer in the main transaction tentatively counts capacity and merges into **active_count** only at the end of the main transaction.

1. **Reserve:** The domain package runs mode-specific admission checks¹. For contained pools, it extends the session-local escrow buffer in the main transaction. It then appends a **reserved** journal entry in an autonomous transaction.
2. **Wait:** The main transaction blocks on the event queue until **CONFIRM**, **CANCEL**, or timeout.
3. **Resolve:** On confirmation, an autonomous **confirmed** journal entry is written and the main transaction commits. On cancel or timeout, a compensating journal entry neutralises the hold and the main transaction rolls back, discarding Δ_w .

Wait phase and event queue. After the autonomous **reserved** write, the same database session subscribes to Transactional Event Queue for a predetermined timeout. It accepts only **CONFIRM** or **CANCEL** messages that match the reservation’s user, resource instance, and context. If no matching message arrives in time, the main transaction treats the hold as timed out, rolls back uncommitted escrow state, and compensates the autonomous journal row. Many sessions can wait concurrently on different correlation tuples. Late or duplicate messages after resolution are harmless because the journal has already moved on.

Figure 4 shows the hold path common to all allocation modes. It leaves out pool rejection before any journal write, exclusivity failures in the autonomous step, and overlap checks on shared resources.

Table 3 lists how journal state, registry $\mathcal{L}$, committed counter base N_{cnt} , escrow buffer Δ_w , and the main transaction look at the end of each phase on a contained pool. The same reservation must not be charged both in journal-derived active count $N_{\mathcal{J}}$ (once **reserved** is visible) and again in escrow buffer Δ_w . Formal invariants are in Section 3.6.

While the hold remains open, each session that extends escrow buffer Δ_w is checked against held capacity:

$$\underbrace{\text{active_count}_{\text{base}}}_{N_{\text{cnt}}(\gamma, \tau)} + \sum_{\text{sessions with buffers}} \Delta_w(\gamma, \tau) \leq \text{total_capacity}. \quad (21)$$

¹Resources can either be requested by instance, by category or if resources are not bound to assets (contained), they are governed by overlap exclusivity

Table 3: Journal current state, active binding registry $\mathcal{L}$, counter base N_{cnt} , escrow buffer Δ_w , and main transaction through the reservation hold on contained pools. Each row is the end state after one phase.

Step	Journal <i>current</i>	Registry $\mathcal{L}$	Base N_{cnt}	Buffer Δ_w	Main tx
Reserve	reserved	insert	unchanged	+1	open
Confirm	confirmed	update	commits +1	ab- sorbed	commit
Cancel / timeout	cancelled	delete	unchanged	dis- carded	rollback

The **RESERVABLE** layer applies this rule when the buffer grows, counting other sessions' in-flight buffers. A violation fails on that attempt, not at main commit. How reported availability relates to this admission check is covered in Section 4.6.

When the main transaction rolls back after an autonomous **reserved** entry has already committed, the journal can still show an active binding although pool counter base N_{cnt} was never incremented because escrow buffer Δ_w was discarded. Compensation appends a release row such as **cancelled**, removes the matching **ActiveAllocation** row, and keeps both rows in the audit trail (Section 4.8). The same recovery applies when a multi-seat loop fails partway through after earlier autonomous commits. A rejected escrow buffer extension needs no journal compensation because no **reserved** write succeeded for that attempt.

4.8 Compensation Framework

Following the formal model in Section 3.7, compensatable lifecycle steps are paired with inverses p^{-1} . Validation covers this pattern on the *reservation hold* path: autonomous **reserved** writes, timeout, confirm, and **CancelReservation/CompensateJournalEntry** recovery (Sections 4.7). That path is the focus of the implementation and workflow integration sections.

Table 4 lists the full airline lifecycle configured for the reference state machine (Figure 3). The first two rows cover the hold window. The remaining rows show how the same inverse pattern extends to check-in, boarding, and completion. Those later steps are abstract in this thesis: the named pairs exist in the package, but no dedicated workflow or test scenario exercises them. They are included to show configurability, not as validated contributions. Terminal completion (**boarded** $\rightarrow$ **completed**) is a normal release transition; **ReopenAllocation** names the administrative inverse (**completed** $\rightarrow$ **boarded**) but is also not a validated contribution.

Each inverse procedure on the hold path and in the extended lifecycle follows the same pattern:

Table 4: Forward procedures and compensating inverses (airline reference configuration)

Forward Procedure	Inverse Procedure	State Transition
<i>Reservation hold (validation focus)</i>		
MakeReservation	CancelReservation	available $\rightarrow$ reserved
ConfirmReservation	UnconfirmReservation	reserved $\rightarrow$ confirmed
<i>Extended lifecycle (configured, not scenario-tested)</i>		
CheckInUser	CancelCheckIn	confirmed $\rightarrow$ checked-in
BoardUser	CancelBoarding	checked-in $\rightarrow$ boarded
CompleteAllocation	ReopenAllocation	boarded $\rightarrow$ completed

- **Validate** that the derived current state permits the inverse transition.
- **Append** a compensating journal row with metadata that marks the entry as compensation.
- **Reverse** pool consumption on the capacity counter when the forward step changed it.

Semantic vs. Exact Reversal

Compensation achieves *semantic equivalence* rather than exact reversal:

$$p^{-1}(p(\mathcal{J}, t), t') \equiv_{\text{semantic}} \mathcal{J} \quad (22)$$

The journal after compensation contains *more* entries than before the original action—both the forward action and its compensation are permanently recorded. This preserves the complete history while restoring the logical state.

Saga Orchestration

Operations that repeat a compensatable step across many instances run inside one main transaction while each journal write commits autonomously. We demonstrate this using **CancelFlight**: a flight-wide release of active bindings. The same pattern appears on the multi-seat reservation path when **AddAllocationJournal** fails mid-loop (Section 4.7).

It illustrates orchestration and recovery under autonomous commits. The forward path loops **CancelReservation** over every active binding on the flight, appending a **cancelled** journal row per instance. It updates context metadata so admission paths reject new allocation attempts on that flight. If the batch fails partway through, the reverse path aborts the flight cancellation and restores the pre-existing state. It rolls back the main transaction and calls **CompensateJournalEntry** for each journal row that already committed. It uses the same autonomous commit pattern as the reservation hold: journal rows

commit immediately while escrow buffers (Δ_w) stay in the main transaction until final commit.

Forward path (success).

1. **Open a savepoint** in the main transaction so escrow buffers and the metadata update can be undone together if a later step fails.
2. **Cancel each active binding on the flight.** The procedure loops over every active allocation on the flight context and calls `CancelReservation`. Each call appends a `cancelled` journal row in an autonomous transaction (Figure 3). Where the binding held pool capacity, the call also extends an escrow buffer for the decrement in the main transaction. We store each new journal id on a local list for possible compensation if the loop aborts.
3. **Record flight cancellation in context metadata.** The `AllocationContext.metadata` row is updated in the main transaction with reason, timestamp, and how many bindings were released. Admission checks consult this metadata and reject new reservations on a cancelled flight. Unallocated seats need no per-instance journal row.
4. **Commit the main transaction.** Escrow buffers merge into `active_count` and the context metadata is updated. The `cancelled` journal rows were already durable from step 2.

Reverse path (failure during the loop or metadata update).

The reverse path aborts an incomplete flight cancellation. Without it, bindings released in earlier loop iterations would stay cancelled even though the operation as a whole failed. Let's assume `CancelReservation` fails partway through the loop. Earlier iterations may already have committed `cancelled` journal rows, while their escrow buffers still sit in the main transaction. We then have to:

1. **Roll back to the savepoint**, discarding all uncommitted escrow buffers and the metadata update.
2. **Compensate each journal id on the local list** by calling `CompensateJournalEntry`. The goal is to reinstate the semantic state before the cancellation attempt, as if the flight cancellation had not touched that instance. The helper appends a compensating journal row, records compensation metadata, and removes the `ActiveAllocation` row introduced by the cancel. If a compensation call itself fails, the helper logs and continues so the original abort reason still reaches the caller. Reconciliation (Section 4.6) handles any unreversed row.
3. **Signal failure to the caller** so the flight cancellation is reported as incomplete.

The two layers are reversed differently. Uncommitted escrow buffers and the metadata update disappear on savepoint rollback because they never merged into the counter base. That is a transaction rollback, not journal compensation. Already-committed **cancelled** rows are neutralised through **CompensateJournalEntry**. The local list of journal ids tracks which autonomous commits must be reversed if the batch aborts.

4.9 Type Substitution System

A request may name a category that cannot cover the full quantity on its own. The package can add substitute categories, place tentative **reserved** holds on each candidate in parallel, and let the caller confirm one offer while the rest are released. *Partial* substitution keeps part of the quantity in the requested category and assigns the remainder to substitutes. *Complete* substitution offers the full quantity only in substitute categories.

CategorySubstitution. The formal substitution relation from Section 3.2 is stored in **CategorySubstitution**. This table is not part of the core ER diagram in Figure 2 because it records domain policy, not runtime allocation state. Each row links a source category to a target category and carries priority, cost adjustment, and an **auto_offer** flag. Rows with **auto_offer** = 'Y' are the domain's predefined counter-offer types when a request names a category that is full or unavailable.

Explicit vs automatic alternatives. Callers can supply substitutes in two ways. **Explicit alternatives** are passed on the request as a list of category names to **MakeReservationWithAlternative**, which offers those types together with any partial availability in the requested category. **Automatic counter-offers** apply when the caller passes no list: the procedure loads eligible **CategorySubstitution** rows for the requested category, ordered by priority. A deployment can therefore define standing upgrade or downgrade paths (for example Economy to Premium or Business) without encoding them in application code.

Offer group and confirmation. Substitution does not add a new lifecycle phase. It follows the same reservation hold pattern as Figure 4. The system writes tentative **reserved** rows, waits for a decision, and then confirms or cancels. The only extension is an *offer group*. Several categories can be held in parallel under one shared **offer_group_id** in journal metadata. Phase 3 is unchanged except that the message includes the accepted offer.

In either explicit or automatic mode, **MakeReservationWithAlternative** opens that group inside one database transaction. It writes **reserved** journal rows for each candidate type

and extends one escrow buffer per category offered. No separate allocation states or queue mechanics are introduced beyond selecting the winning offer.

Remark. Substitution has been verified at the system-design and PL/SQL level (Appendix: Implementation Reference, scenarios S2–S3). It is not integrated with a working APEX Workflow demo in the current implementation.

4.10 Validation

We verified our concept through self-contained SQL*Plus and SQLcl validation scripts. Each scenario creates isolated fixtures, runs a scripted flow, and queries `AllocationJournal`, `ActiveAllocation`, and `Capacity` before tearing down. This is qualitative correctness validation, not performance benchmarking. Pseudocode sketches and the full scenario index appear in Appendix: Implementation Reference.

Table 5: Core validation scenarios

ID	Scenario	Formal / design claim	Sess.	Pass criterion
V1	Instance reservation and confirmation	Hold window, journal lifecycle	2	<code>reserved</code> $\rightarrow$ <code>confirmed</code> , <code>active_count</code> +1
V2	Double-booking rejection	Exclusivity on $\mathcal{L}$	2	ORA-20604 before queue wait. No second journal row
V3	Hold-timeout auto-cancellation	Timeout recovery (P3)	1	<code>cancelled</code> rows with timeout metadata. Capacity restored
V4	Shared-resource interval conflict	Shared-mode temporal exclusivity	2	ORA-20703 on overlap attempt
V5	Concurrent pool admission and compensation	Concurrent admission + compensation	2	One winner. Loser gets compensating <code>cancelled</code> rows or ORA-20001

These scenarios exercise the transaction strategy (Section 4.7), constraint enforcement (Section 3.5), and compensation framework (Section 4.8).

Worked example for double booking rejection (V2). Table 6 illustrates journal and capacity state when a second user targets a seat already held by the first user. User A reserves and confirms seat M1 successfully. User B then attempts to reserve the same seat in the same context. The second attempt fails at admission with ORA-20604 because `ActiveAllocation` already binds M1 in that context.

Table 6: Journal state during double booking rejection (V2, seat M1 in one context)

Step	Journal status (M1)	active_count
After User A reserves M1	reserved	0
After User A confirms M1	confirmed	1
User B attempts M1	(no new row)	1

The committed counter increments only on confirm (Section 4.7), so `active_count` remains 1 throughout User B’s failed attempt. The exclusivity check runs inside *AddAllocationJournal* before the hold window opens (Appendix: Implementation Pseudocode).

Limitations. Most happy-path scenarios require two database sessions. Each script embeds explicit pass/fail assertions on the resulting journal, registry, and counter state, but the scripts are run by hand rather than as a continuous-integration suite, and no load benchmark is included. Explicit substitution offer (S2), Partial substitution offer (S3), and Maintenance overlap blocking (S4) were exercised manually but are excluded from Table 5 because substitution is not wired to the APEX workflow demo. Scenario S4 validates Path A and Path B from Section 4.5. The full index is Table 15 (Appendix: Validation Scenario Index).

Together, scenarios V1–V5 support the feasibility claim from the Introduction: core invariants of the journal and escrow design hold.

The design trade-offs above are intentional. Append-only journal and typed escrow counters buy auditability and lock-free admission at the cost of storage growth and Oracle-specific deployment. Domain vocabulary, state machines, and substitution rules stay configurable per deployment while the hold, journal, and compensation core stays invariant. Airline seating stress-tests that split in one demanding setting. The same machinery is intended to carry other domains that book resources for a time interval without changing the package boundary described in Section 4.2.

5 Workflow Integration

Section 4 built the database core: capacity management, conflict detection, compensation, and error handling. This section shows what a workflow designer actually calls, why one step must wait on a message queue, and how the same plugin covers flight seat booking and worker invitation to meetings.

Within the workflow environment, designers do not need to implement locking, compensation, or error recovery. They call a small custom plugin that forwards intentions to the resource package and waits for the outcome. The workflow reacts to that outcome. The database handles the rest.

Nothing new is added to the resource model here. APEX Workflow simply exercises the same entry points as the SQL*Plus validation scripts. Those entry points include reserve with hold, confirm or cancel via the queue, and synchronous lifecycle steps thereafter.

What follows is an *initial demonstration*, not a production-ready integration. Terminal sessions (SQL*Plus/SQLcl) served as the primary validation method during development. The APEX Workflow paths shown here cover the airline seat booking and meeting-invite scenarios.

The sections below move from motivation to mechanism. They cover why workflows delegate resource logic to the database, what constraints the runtime imposes, how the hold window works, and how APEX realises it.

A note on terminology. In this section, *event* refers only to an explicit queue message in the reservation hold. It does *not* mean that every workflow step emits a message. *Implementation* and *our concept* refer to the Oracle instantiation described in Sections 3 and 4. The implementation uses Transactional Event Queues (TxEventQ) only for the reservation hold window (waiting for human confirm/cancel with a timeout). Every other activity calls the resource package synchronously.

5.1 Integration at the Process Boundary

Section 4 defined tables, packages, and hold semantics internally. Here we ask what crosses the workflow boundary.

Specifically, it addresses:

- How does a workflow activity request a resource?
- How does a human approval or rejection reach the database?
- What happens when the passenger takes too long to decide?
- How does compensation work when something fails?

The core objective is to show that a workflow designer can bind resources to process steps through a small plugin surface, without reimplementing admission or recovery in workflow scripts. The key lifecycle transition is **reserved** $\rightarrow$ **confirmed** or **cancelled**. This must work correctly even when a human decision takes minutes, or never arrives at all.

Workflow engines are good at control flow: “do A, then B, then if X do C else D.” But they typically treat resources as black boxes. BPMN 2.0 models resources as role assignments (“this task is performed by a manager”), not as bookable entities with capacity limits and temporal constraints (Object Management Group, 2011; Russell et al., 2005; Dumas et al., 2018).

When a workflow needs to reserve a seat, check capacity, handle timeouts, or undo a failed booking, the engine usually delegates to an external service. That service must reimplement compensation, concurrency control, and audit logging that could live in the database instead. This is the same gap that advanced transaction models for workflow contexts set out to close (Alonso et al., 1996).

What the integration contributes. Our concept wires the resource package into Oracle APEX Workflow through a custom plugin. A flight booking must survive minutes of human delay during the reserve step. The confirm or cancel action must reach the same database session that opened the hold. If no decision arrives, the seat must be released without manual cleanup. A second workflow invites workers to existing meetings and exercises overlap rejection on shared people resources. Both paths call the same plugin with different inputs. The database chooses pool admission or overlap checking internally.

Table 7 contrasts that behaviour with typical BPMN resource modelling. The escrow and journal details are in Section 4.7. The focus here stays on what crosses the process boundary.

5.2 Implications for Workflow Modelling

For a workflow designer, the practical consequence is narrow process models. Activities name an intent such as **RESERVE**, **CHECKIN**, or **CANCEL**. They supply context, user, and either a resource instance or a category with quantity. Capacity checks, journal writes, overlap rejection, and compensation stay inside the resource package described in Section 4. Process models stay thin because booking rules live in one place. The journal remains even if workflow history is purged. Most steps are synchronous PL/SQL calls through the plugin, except the human hold window described next.

5.3 Integration Constraints Imposed by the Workflow Runtime

Why not just call a PL/SQL procedure for every workflow step? Because some steps involve waiting for humans, and humans are slow.

The design must handle:

1. **Human decision latency.** A passenger might take several minutes to confirm or cancel a reservation. The seat must stay reserved during that time, but not forever.
2. **No long-held row locks.** We cannot keep a transaction with row locks open while waiting for a human. Instead, the session blocks on a queue, waiting for a decision message (or timeout) without locking resource rows.
3. **Failure recovery.** If the confirmation step fails after the seat was reserved, the reservation must be cancelled automatically.
4. **Mixed resource types.** The same plugin must work for pool-based resources (flight seats) and overlap-based resources (workers invited to meetings).
5. **Traceability.** Every reservation attempt that the system allows must be journalised.

The solution: a hybrid approach. Most workflow activities call PL/SQL procedures synchronously. They complete immediately.² The reservation hold window uses a message queue while waiting for human confirmation. The workflow publishes a decision. The database session waiting on that reservation receives it and acts. If no decision arrives within the timeout, the database treats it as a cancellation.

Table 7 summarises how this approach differs from typical workflow-engine support.

Table 7: Comparison: standards/engines vs. this integration approach

Aspect	BPMN/Engine Support	This Thesis
Resource modelling	“Resource” = who performs tasks (roles, departments)	“Resource” = bookable entity with lifecycle (seats, workers)
Conflict detection	External service responsibility	Database enforces capacity and overlap rules
Timeout behaviour	Generic engine timer, workflow handles result	Timeout triggers automatic cancellation and compensation
Compensation	Workflow must orchestrate undo logic	Database knows inverses and workflow reacts

²Currently simplified procedures, like checkin or boarding, which were not modeled in detail.

5.4 Workflow-Coupled Resource Execution Model

In Section 4.7 we defined the hold transaction. It extends the escrow buffer, writes `reserved` in an autonomous transaction, waits on the queue, and then confirms or cancels. Figures 5 and 6 show the same path from the workflow side. They show who calls `RESERVE`, where the human task sits, and how `CONFIRM` or `CANCEL` reaches the waiting session.

Phase 1: Reservation Request When a workflow activity calls `RESERVE`, the plugin forwards context, user, target instance or category, and timeout to `MakeReservation` (Section 4.7). The database session then blocks on the queue until `CONFIRM`, `CANCEL`, or timeout. The workflow activity does not return before then. The seat is reserved while the process waits for a human decision.

Phase 2: Human Decision While the database session waits, the workflow presents a human task. In our implementation this is a simple accept or reject step. In a production process the same slot could hold a payment form or any other service. The workflow only needs to publish `CONFIRM` or `CANCEL` to the queue when the user decides. The message carries a correlation identifier that links it to the pending reservation. The queue delivers it only to the session waiting for that combination.

Phase 3: Completion, Timeout, or Compensation The waiting database session eventually receives one of three outcomes:

- **CONFIRM received:** The session updates the journal to `confirmed` and commits. The seat is now booked.
- **CANCEL received:** The session appends `cancelled` journal entries and rolls back. The seat is released.
- **Timeout (no message):** The session treats this as an implicit `CANCEL`. The seat is released automatically.

In all three cases, the journal records what happened. Further failure recovery follows the compensation framework in Section 4.8.

State Alignment Table 8 summarises the state transitions during the hold window. The workflow and the database always agree on resource state at each step.

Table 8: State transitions during the reservation hold window

What happens	Before	After	Note
Passenger selects seat	available	reserved	Hold window starts
Passenger confirms	reserved	confirmed	Seat is booked
Passenger cancels	reserved	cancelled	Seat is released
Timeout expires	reserved	cancelled	Automatic release

What happens after confirmation? Once the hold window closes, the remaining lifecycle is straightforward. Check-in, boarding, and completion do not need the queue-based hold window because they complete immediately. The workflow calls `CHECKIN`, `BOARD`, or `COMPLETE`, and the database updates the journal synchronously.

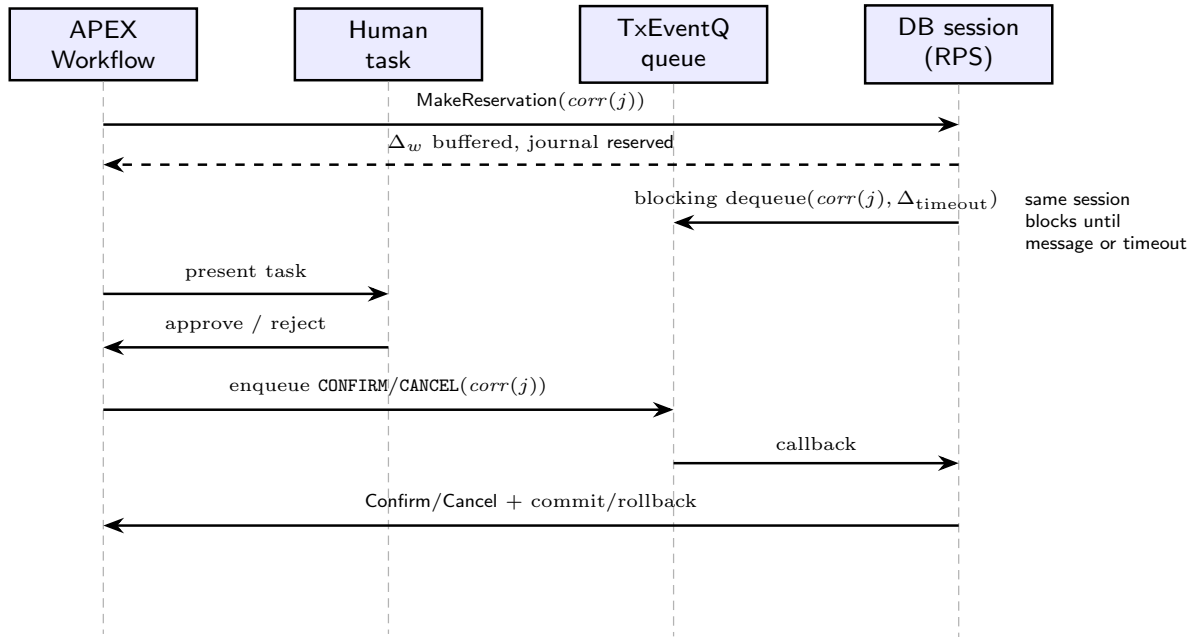


Figure 5: Workflow-to-resource sequence

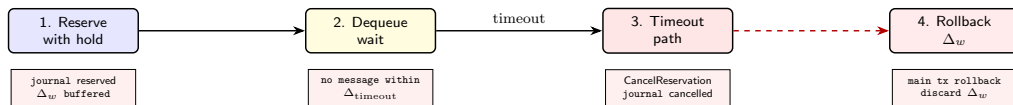


Figure 6: Timeout and compensation flow

5.5 Reference Workflow: Flight Booking

The abstract three phases become concrete in the flight booking workflow built in the APEX designer.

The scenario. A passenger wants to book a seat on flight LH123. The designer workflow supports reservation by specific seat instance or by category and quantity in the same process model. Figure 7 shows the main path for instance booking (seat 12A). The steps are:

1. **Select flight and target.** The passenger picks LH123 and either seat 12A or a category with quantity (for example two Economy seats). No database state changes yet.
2. **Reserve with hold.** The workflow calls the plugin with `p_action=RESERVE`. This follows Phase 1 in Figure 6: admission check, **reserved** journal entry, and blocking wait on the queue. The workflow activity does not return yet.
3. **Confirm hold (human task).** The workflow presents a simple accept or reject task. Accept publishes **CONFIRM** to the queue. Reject publishes **CANCEL**. This corresponds to Phase 2 while the database session blocks.
4. **Resolution.** The database session receives the decision and acts as in Phase 3:
 - On **CONFIRM**, it appends a **confirmed** journal entry and commits the main transaction.
 - On **CANCEL** or timeout, it appends a **cancelled** journal entry and rolls back the main transaction.

Now the **RESERVE** activity returns with the outcome.

5. **Check-in (human task).** If confirmed, accept calls the plugin with **CHECKIN**. Reject calls **CANCEL**. Both complete synchronously.
6. **Board (human task).** Accept calls **BOARD**. Reject calls **CANCEL**.
7. **Complete (human task).** Accept calls **COMPLETE**. Reject calls **CANCEL**.

Category mode. The same workflow passes category and quantity instead of an instance id to **RESERVE**. The database allocates from the pool or may offer substitution alternatives (Section 4.9). The plugin interface is unchanged. Substitution is an API capability in Section 4. It was not wired as a separate APEX path in the current implementation.

Design note on human tasks. Human steps between plugin calls are ordinary workflow configuration. The implementation uses minimal accept or reject tasks because lifecycle UX was not the research focus. A production process could insert payment gateways, forms, or approvals without changing the plugin contract. Check-in, board, and complete are deliberately abstract for the same reason. The contribution is the plugin surface that binds resources and advances lifecycle states with simple API calls, not the shape of intermediate human tasks.

Key observations.

- Only the reserve step (steps 2–4) uses the queue. All lifecycle steps thereafter are synchronous plugin calls.
- The journal records the full history: reserved → confirmed → checked-in → boarded → completed. This audit trail exists in the database, independent of the workflow engine.

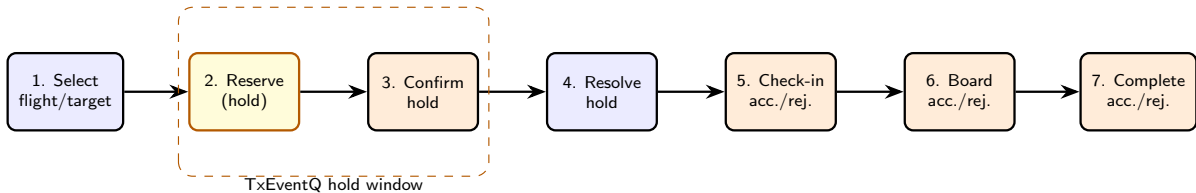


Figure 7: Flight booking workflow in the APEX designer. Steps 5–7 are minimal accept or reject human tasks that call synchronous plugin actions.

5.6 Reference Workflow: Inviting Workers to Meetings

The flight walkthrough used contained pool booking. The second APEX workflow exercises overlap checking on shared people resources.

Workers are not counted like seats on a flight. The same person can attend several meetings on one day if the intervals do not overlap. The constraint is temporal overlap across allocation contexts. If a worker is already planned for a conflicting meeting, a new invitation must fail.

How the demonstration workflow handles this. The designer workflow starts from existing meetings and people. The organizer selects a meeting context. The UI lists workers as available or busy for that meeting interval. That list is assistive. It helps the organizer choose, but it is not authoritative. The plugin call that binds the worker to the meeting is authoritative. The database checks overlap across all contexts and returns success or `TIME_CONFLICT`. See Figure 8.

Unlike the flight workflow, this path does not use the hold window. The organizer binds a worker in one step, so the plugin call returns once the database has checked overlap. The queue is reserved for cases where human delay sits between reserve and resolve (Section 5.5). This workflow is a second demo. It shows overlap rejection at the process boundary, not a complete invite-and-accept lifecycle.

Layered correctness. For worker invitation, a sensible UI would:

1. Let the organizer pick a meeting first
2. Show which workers are free during that meeting interval (assistive correctness)
3. Let the organizer pick a worker
4. Call the plugin to bind the worker. The database enforces the constraint authoritatively.

Even if the UI shows a worker as available, another organizer might invite that worker first. The database rejects the second attempt with `TIME_CONFLICT`.

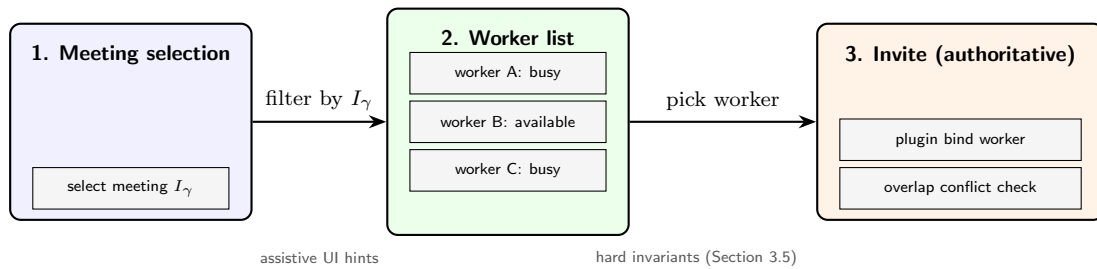


Figure 8: Worker invitation workflow. The UI shows availability hints for the meeting interval. The plugin call enforces overlap rules authoritatively.

5.7 Plugin Design and Behavior

So far the integration was described in technology-neutral terms. This section maps it to Oracle APEX Workflow:

- **Oracle APEX Workflow** as the process engine
- **Transactional Event Queues (TxEventQ)** as the message queue for the hold window
- **PL/SQL packages** for resource logic

Everything runs inside the database. No external orchestrator or message broker is required.

The pattern itself is not Oracle-specific. Any system with correlated messaging and blocking waits could implement the same flow.

Table 9: Custom plugin parameters

Parameter	Role
p_action	Intent: RESERVE, CHECKIN, BOARD, COMPLETE, CANCEL
Context id	Allocation context (flight, meeting, ...)
User id	Requesting subject
Instance id <i>or</i> category + quantity	Instance mode vs pool mode for RESERVE
Timeout	Hold window for RESERVE only

Plugin inputs. The custom plugin activity takes an intent and the booking fields the resource API needs. Table 9 lists the parameters a designer configures per activity.

On success the activity returns normally. On failure the plugin surfaces structured outcomes that the workflow branches on with `NATIVE_WORKFLOW_SWITCH`. The designer does not write compensation scripts. The plugin wraps a subset of the public procedure surface. The grouped API catalogue is in Appendix: Public API Surface.

Workflow Vocabulary. Workflow activities use a simple vocabulary:

RESERVE	Create a reservation (instance or category mode)
CHECKIN	Mark the passenger as checked in
BOARD	Mark the passenger as boarded
COMPLETE	Close the allocation after the flight
CANCEL	Cancel the reservation (with optional reason)

Maintenance blocking. Administrative maintenance propagation (Section 4.5) runs without workflow involvement. Designers do not need to orchestrate individual blocked journal entries in the process model.

What Happens When Reservation Fails? Not every RESERVE succeeds. The plugin translates database errors into workflow-visible outcomes:

TIME_CONFLICT	Worker already allocated in an overlapping meeting
CAPACITY_EXHAUSTED	No seats left in that category
INSTANCE_UNAVAILABLE	Instance is intrinsically unavailable, or journal-blocked / maintenance-blocked in the target context
ALREADY_ALLOCATED	User already has this seat reserved

The workflow can branch on these outcomes (e.g., offer alternatives if `CAPACITY_EXHAUSTED`) without knowing internal error codes. A fuller error catalogue with validation scenario cross-references is Table 14 (Appendix: Application Error Catalogue). Workflow designers never see the message queue. When they call `RESERVE`, the plugin handles the journal write, queue wait, timeout, and confirmation inside one activity. The queue is an implementation detail of the hold window, not something designers should model.

How Concepts Map to APEX Artifacts. Table 10 shows exactly which APEX components implement each concept.

Table 10: Concepts mapped to APEX Workflow artifacts

Concept	APEX Implementation
Reserve (hold)	Plugin: <code>p.action=RESERVE</code> (queue wait internal)
Confirm hold	<code>NATIVE_CREATE_TASK</code> accept/reject → enqueue <code>CONFIRM/CANCEL</code>
Lifecycle human tasks	<code>NATIVE_CREATE_TASK</code> accept → plugin <code>CHECKIN/BOARD/COMPLETE</code> ; reject → <code>CANCEL</code>
Branch on outcome	<code>NATIVE_WORKFLOW_SWITCH</code> on plugin result
Worker invite	Plugin synchronous bind (no hold window)

5.8 Summary

Workflow designers interact with the resource core through the custom plugin. They pass an intent and the necessary booking parameters. The workflow branches on the outcome. Apart from the reserve hold window in the flight workflow, every step is a synchronous plugin call. The queue appears only where human latency spans the reserve step.

The implementation is an initial concept, not a production system. Validation relied on SQL*Plus/SQLcl scripts and two APEX Workflow scenarios (flight booking and worker invitation). Human tasks in the implementation are minimal accept or reject steps. Designers can model those steps differently. A real airline process might add payment gateways or fraud checks outside the plugin boundary.

Production concerns such as SLAs, multi-tenancy, enterprise governance, and portability to non-Oracle engines are deferred to future work (Section 6).

6 Future Work

6.1 Scope and Framing

The original target of this work was to develop an *intuition* for whether and how heterogeneous resources can be promoted to first-class database citizens, and to demonstrate *feasibility* end-to-end on a representative use-case (airline seat allocation). A production-grade implementation was never the intended deliverable, and the scope delimitation in Section 1 reflects that framing.

In the course of the work, the artifacts produced went beyond a pure intuition study. The thesis contributes a formal model unifying typed capacity escrow (Section 3) with per-instance lifecycle (**C1**), temporal allocation within contexts on the database layer (**C2**), a journal-with-inverse compensation framework (**C3**), and an executable workflow-integration semantics (**C4**), together with a conceptual domain-level interface (Section 4) that moves transaction strategy into the database and shifts admission, consistency and recovery away from developers and process designers. The formal model gives a shared vocabulary: The append-only journal and compensating procedures define how failed steps are reversed while the audit trail is preserved. The workflow integration shows how business processes can interact with that layer.

For further contributions, several directions can be pursued *in parallel*. Themes such as a richer constraint layer, operational research and nested or distributed sagas reflect ongoing supervisory discussion and this section does not claim a complete picture of the steps required to turn them into full research programmes.

6.2 Research Directions and Extensions

The following introduces separate *exploratory research themes* and *engineering and integration work*, where open questions are still at large, but might lead to desirable contributions. Both can advance independently on the same foundation.

Constraint-Expressive Resource Selection We answer requests of the form “allocate one instance of resource type τ in allocation context γ ”, optionally widened by type substitution. That is enough for airline seating, but in many domains the interesting problem is to select which instance, among those satisfying a body of constraints and ranked by a preference structure, is best suited to the recipient.

A natural follow-up could be to move toward declarative hard and soft criteria over the journal, rather than the small invariant set currently employed. Whether such a layer needs its own query surface, how it would interact with lock-free capacity semantics, and how temporal predicates over an append-only journal would be kept compliant are

open. This thesis does not specify that language or its semantics; it only argues that the present substrate could support exploring them.

Objective-Driven Allocation and Operational Research Another possible direction is to treat allocation as an optimisation problem in the sense of operational research: among admissible candidates, select those that score well on utilisation, substitution cost $c(\tau_i, \tau_j)$, fairness or service-level goals. Classical resource-constrained scheduling offers useful machinery (Brucker, 2007; Pinedo, 2016), but it is unclear how much of it transfers unchanged once reservations must remain revocable, holds may disappear through timeout or compensation, and “undo” is journal-backed rather than a simple counter rollback.

Nested and Distributed Sagas Today’s compensation model is intentionally flat: a saga runs over one journal, and failure triggers inverse in reverse order. Real processes may span several resource domains, each with its own journal. Supervisor guidance has pointed toward *nested* or *distributed* sagas as a longer-horizon question: how outer cancellation is composed per-domain, and how audit trails could still be linked across boundaries.

Whether hierarchical rollback can preserve the semantic guarantees this thesis established for a single journal, and what failure semantics would look like when a participant is unreachable, would need careful formal work that lies outside the scope of the present thesis. The flat compensation is a deliberate starting point, not a claim that multi-domain composition is solved.

Further Engineering and Integration

Here we propose extensions of the current system, which may provide a sensible evolution of the current functional boundary.

Delay propagation and rescheduling. We deliberately excluded automatic delay propagation and cascading rescheduling. Extending the model would mean defining how a disruption in one allocation context propagates constraints or compensations to related contexts without breaking journal consistency. A delayed flight, for example, can force changes to connecting bookings. As a planned asset is not available, either alternatives need to be found - with their own constraints like higher or lower passenger count - or passengers have to be transferred or reimbursed. A next step could be to enable the database to coordinate related updates based on a chosen objective function - minimizing passenger cancellations and compensation, cost or amount of subsequent rescheduling.

Event notification. The current version uses Transactional Event Queues only for the reservation hold window (Figure 1, dashed path not implemented). The same journal

that supports audit could feed a broader notification layer—operators, dashboards or services get notified when, for example, a high-value allocation is cancelled or capacity approaches saturation. A natural shape would be context- or transition-filtered subscriptions (e.g. on **cancelled** entries in a given allocation context) delivered asynchronously once the enclosing transaction commits. That separation would let customer services react to committed journal facts without polling. The narrow TxEventQ path is implemented, but would benefit strongly from a more in-depth leverage.

Multi-domain case studies and portability. Domain neutrality has been incorporated by design, but only the airline seating use-case is exercised end-to-end. Instantiating at least one further domain would test that claim further. Re-implementing on a second RDBMS would separate Oracle-specific choices (TxEventQ, selective dequeue, packaged procedures) from what the formal model actually requires.

The present thesis is a *starting point*: an executable feasibility study and a formal vocabulary, not a closed product or a fully specified research programme. Engineering extensions follow most directly from limitations documented in Sections 3–5; the research themes above would need their own scoping, supervision and literature grounding before they could be pursued as standalone contributions.

7 Conclusion

7.1 Summary of Contributions

This thesis set out to develop intuition for, and demonstrate the feasibility of, promoting heterogeneous resources to first-class database citizens while preserving the concurrency properties of escrow. What follows compresses the result into four themes.

Extending escrow. Classical escrow as surveyed in Section 2.1 and formalised in Section 3 excels at lock-free reservation on aggregate counters but cannot express individual identity, lifecycle or compensation. Rather than replacing escrow, the thesis extends it along two cooperating layers: typed capacity counters retain the original admission semantics, while an orthogonal allocation journal tracks each resource instance. High-concurrency pool reservation and per-instance lifecycle management therefore coexist in one model.

Resources as first-class objects. Resource types, contexts, instances, state machines and audit trails are encapsulated in the database layer. They are implemented as schema, journal and packaged procedures rather than re-encoded in application or workflow logic. The airline seat allocation scenario is a reference instantiation chosen for its complexity. The architecture itself is intended as a general-purpose framework for any domain that allocates named resources for a time interval.

Workflow integration. Section 5 demonstrates that a workflow engine can orchestrate reservation lifecycles through hold, human decision, timeout, and compensation by calling into that database layer. Transactional queues appear only where asynchronous human latency requires them. Human steps stay in the workflow. Legality, journal updates, and recovery stay in the database package. This closes a gap left by mainstream BPMN and workflow resource patterns, which describe organisational assignment but not executable, compensable reservation semantics.

Responsibility shift. The deepest practical lesson of the work is architectural rather than domain-specific. Admission rules, overlap checks, escrow buffering, timeout cancellation and saga-style recovery are handled inside the database boundary. Process designers see a small stable procedure surface. Error handling and compensation need not be re-implemented in workflow scripts or external services for each new edge case. They follow systematically from procedure-inverse pairs and journal replay. A production-grade system was never the target. The target was intuition and feasibility. That target was met. The project additionally produced a concrete structural skeleton and DML for context, enough to make the research directions of Section 6 actionable rather than speculative.

Together, these results point to a clear division of labour between the resource core and the workflow engine, which the Reflections below revisit in more detail. Feasibility is

supported by manual scenario validation (Section 4.10), with remaining scope boundaries listed under Limitations below.

7.2 Limitations

We deliberately scoped our claims. The following limitations follow from that framing and from the initial, non-production nature of the implementation.

- The formal model includes only capacity, exclusivity, type, and intrinsic-admission invariants. It does not provide a general constraint or preference language, nor atomic multi-context bookings such as connecting flights.
- Escrow semantics apply to contained, asset-bound pools only. Shared allocation relies on journal overlap and exclusivity without typed capacity counters.
- Workflow integration is an initial demonstration, not a production-ready integration. Category substitution is implemented in the database package but not wired to the APEX workflow demo.
- Only airline seat allocation is exercised end-to-end. Domain neutrality is a design goal that a second domain would need to validate further.
- The implementation depends on Oracle-specific platform features, including lock-free reservation and Transactional Event Queues.
- Reconciliation is a scheduled administrative path. Residual drift remains possible when compensation itself fails or a hold ends without a clean cancel or confirm.
- Manual scenario scripts validate core invariants (Section 4.10) with explicit pass/-fail assertions on journal, registry, and counter state, but they are executed by hand rather than as a continuous-integration suite, and no load testing is reported.

Section 6 discusses extension paths that address several of these boundaries.

7.3 Reflections

The architectural choice to keep resource semantics inside the database transactional boundary matches the direction set in supervision and related work. This thesis does not claim to have discovered that principle in isolation. It instantiates it through typed escrow counters, an append-only allocation journal, hold semantics, and compensating inverses in one reference model, with manual validation of the core invariants.

Building that model nonetheless sharpened what the boundary must contain. Admission, overlap checks, timeout recovery, and saga-style rollback need shared journal and counter rules if workflows are to stay thin. Compensation outside the database must be reimplemented for each failure mode. Journal-backed inverses and escrow admission

therefore belong in the same transactional layer as the journal itself, not in process scripts or external services.

The present implementation is a feasible starting point for further research and engineering, not a production deployment. Whether the two-layer structure retained here will survive unchanged when integrated into a running product remains open. A concrete test will be to merge with the platform's lock-free reservation path on Oracle **RESERVABLE**: the formal split between aggregate counters and per-instance lifecycle is deliberate, but production packaging may favour tighter coupling or different commit boundaries.

What this work argues with more confidence is narrower. Resource semantics belong in the database transactional boundary. Workflows should sequence decisions and call a stable procedure surface. Escrow remains the right substrate for pool admission once extended with instance-level journal and compensation, rather than replaced. Section 6 outlines how that skeleton might evolve once engineering plans move from feasibility study toward a running system.

Acknowledgements

I am grateful to Dieter Gawlick, Vasudha Krishnaswamy, and Ralf Müller for the many discussions that shaped this work and for generously sharing their experience with escrow transactions and Oracle Database internals. I also thank my supervisor, Prof. Matthias Weidlich, for his guidance and feedback throughout the project. Any remaining errors are my own.

References

- James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi: 10.1145/182.358434.
- Gustavo Alonso, Divyakant Agrawal, Amr El Abbadi, Mohan Kamath, Roger Günthör, and C. Mohan. Advanced transaction models in workflow contexts. In *Proceedings of the Twelfth International Conference on Data Engineering (ICDE)*, pages 574–581. IEEE, 1996. doi: 10.1109/ICDE.1996.492208.
- Dominik Bernhardt, Matthias Weidlich, Ralf Müller, and Dieter Gawlick. Study project: Foundations of a resource planning system. Technical report, Humboldt-Universität zu Berlin, December 2024. Study project.
- Peter Brucker. *Scheduling Algorithms*. Springer, 5th edition, 2007. ISBN 978-3540695158.
- Panayiotis K. Chrysanthis and Krithi Ramamritham. ACTA: A framework for specifying and reasoning about transaction structure and behavior. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*, pages 194–203. ACM, 1990. doi: 10.1145/93597.98730.
- Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*. Springer, 2nd edition, 2018. ISBN 978-3-662-56509-4.
- Martin Fowler. Event sourcing. <https://martinfowler.com/eaDev/EventSourcing.html>, 2005. Accessed: 2026-07-03.
- Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 249–259. ACM, 1987. doi: 10.1145/38713.38742.
- Dieter Gawlick and David Kinkade. Varieties of concurrency control in IMS/VS fast path. *IEEE Database Engineering Bulletin*, 8(2):3–10, 1985.
- Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993. ISBN 978-1558601901.
- Pat Helland. Immutability changes everything. *Communications of the ACM*, 59(1): 64–70, 2015. doi: 10.1145/2844113.
- Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004. doi: 10.2307/25148625.
- Shuai Lu, Sihang Liu, Shuai Xia, Feng Zhang, Rongrong Gu, Haibo Chen, and Binyu Zang. Ad hoc transactions: What they are and why we should care. *Communications of the ACM*, 66(10):76–85, 2023. doi: 10.1145/3604432.

- Object Management Group. Business process model and notation (BPMN) version 2.0. Technical Report formal/2011-01-03, OMG, 2011. URL <https://www.omg.org/spec/BPMN/2.0/>. Accessed: 2026-07-03.
- Patrick E. O’Neil. The escrow transactional method. *ACM Transactions on Database Systems*, 11(4):405–430, 1986. doi: 10.1145/7239.7265.
- Oracle Corporation. *About Workflows*, 2026a. URL <https://docs.oracle.com/en/database/oracle/apex/26.1/htmdb/about-workflows.html>. Oracle APEX 26.1 documentation. Accessed: 2026-07-03.
- Oracle Corporation. *Using Lock-Free Reservation*, 2026b. URL <https://docs.oracle.com/en/database/oracle/oracle-database/26/adfns/using-lock-free-reservation.html>. Oracle Database 26ai documentation, Application Developer’s Guide. Accessed: 2026-07-03.
- Oracle Corporation. *Introduction to Transactional Event Queues and Advanced Queuing*, 2026c. URL <https://docs.oracle.com/en/database/oracle/oracle-database/26/adque/aq-introduction.html>. Oracle Database 26ai documentation. Accessed: 2026-07-03.
- Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5th edition, 2016. ISBN 978-3319265780.
- Luise Pufahl, Fabian Stiehle, Sven Ihde, Mathias Weske, and Ingo Weber. Resource allocation in business process executions—a systematic literature study. *Information Systems*, 132:102541, 2025. doi: 10.1016/j.is.2025.102541.
- Nick Russell, Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, and David Edmond. Workflow resource patterns: Identification, representation and tool support. Technical Report BPM-05-01, BPM Center, 2005. URL <http://www.workflowpatterns.com/patterns/resource/>. Accessed: 2026-07-03.

Appendix

Mathematical Notation

Table 11: Mathematical notation used in the formal model

Symbol	Meaning
Sets	
$\mathcal{T}$	Set of resource types
$\mathcal{R}$	Set of resource instances
$\mathcal{A}$	Set of resource assets
Γ	Set of allocation contexts
$\mathcal{J}$	Allocation journal (sequence of entries)
$\mathcal{J}_\gamma$	Journal entries in allocation context γ
$\mathcal{L}$	Active binding set; elements are (context γ , instance r) pairs; implementation table ActiveAllocation (within-context enforcement)
$\mathcal{S}$	Substitution relation ($\mathcal{S} \subseteq \mathcal{T} \times \mathcal{T}$)
$\mathcal{U}$	Set of recipients (users)
$\mathcal{Q}$	Set of resource requests
$\mathcal{P}$	Set of domain procedures
$\mathcal{E}$	Set of lifecycle events (confirm, cancel, timeout, ...)
$\mathcal{W}$	Set of concurrent transactions
Σ_{inst}	Set of intrinsic states (e.g. available , unavailable)
Σ_{alloc}	Set of allocation states
Σ_{active}	Active allocation states (journal entry holds the binding)
Σ_{release}	Release allocation states (journal entry terminates the binding)
$\mathbb{T}$	Time domain ($\mathbb{T} = \{t \in \mathbb{R} \mid t \geq 0\}$)
$\mathbb{N}$	Natural numbers
$\mathbb{R}$	Real numbers
Variables	
τ	Resource type ($\tau \in \mathcal{T}$)
r	Resource instance ($r \in \mathcal{R}$)
a	Resource asset ($a \in \mathcal{A}$)
γ	Allocation context ($\gamma \in \Gamma$)
j	Journal entry ($j \in \mathcal{J}$)
u	Recipient/user ($u \in \mathcal{U}$)
t	Time point ($t \in \mathbb{T}$)
I	Time interval ($I = [t_s, t_e]$)
q	Resource request ($q \in \mathcal{Q}$)
<i>continued on next page</i>	

Table 11 – *continued from previous page*

Symbol	Meaning
p	Domain procedure ($p \in \mathcal{P}$)
σ	State (allocation or intrinsic)
μ	Metadata
$\perp$	Null/undefined value
Functions	
$type(r)$	Maps instance r to its type
$asset(r)$	Maps instance r to its containing asset
$\sigma_{\text{inst}}(r)$	Current intrinsic state of instance r (snapshot; checked at admission)
$\sigma_{\text{alloc}}(j)$	Allocation state of journal entry j
$\eta(\tau)$	Hierarchy level of type τ
$K(a, \tau)$	Capacity of asset a for type τ
$A(\gamma, \tau)$	Available capacity (contained contexts); committed snapshot: $K - N_{\mathcal{J}}$; for admission use $K - H(\gamma, \tau, t)$
$N_{\mathcal{J}}(\gamma, \tau)$	Journal-derived active bindings for type τ in γ
$N_{\text{cnt}}(\gamma, \tau)$	Committed counter base (used in H during concurrency)
$H(\gamma, \tau, t)$	Held capacity: $N_{\text{cnt}} + \sum_w \Delta_w$
$\Delta_w(\gamma, \tau)$	Escrow buffer for transaction w (session-local RESERVABLE increment until main commit)
$c(\tau_i, \tau_j)$	Cost adjustment for substitution $\tau_i \rightarrow \tau_j$
δ	Partial transition relation ($\delta \subseteq \Sigma_{\text{alloc}} \times \mathcal{E} \times \Sigma_{\text{alloc}}$)
$start(I)$	Start time of interval I
$end(I)$	End time of interval I
$overlaps(I_1, I_2)$	Predicate: intervals I_1 and I_2 overlap
$current(\gamma, r)$	Derived availability/allocation status for context γ , instance r
$\lambda(\gamma, r)$	Journal entry id for the active binding on (context γ , instance r) pair
$subtype(\tau_i, \tau_j)$	Type of substitution (upgrade/downgrade/lateral)
$compensate(S, i)$	Compensation sequence for failed saga S at step i
Relations and Operators	
$\preceq$	Type hierarchy ordering
$\prec$	Strict type hierarchy ordering
$\equiv_{\text{semantic}}$	Semantic equivalence
p^{-1}	Inverse (compensating) procedure for p

Implementation Reference

This appendix summarises how the implementation maps the formal model (Section 3) to Oracle schema and packaged procedures (Section 4). It provides a schema mapping, a grouped public API surface, pseudocode sketches of core mechanisms, application errors tied to validation scenarios, and the full scenario index.

Code and Data Availability

The database schema, PL/SQL packages, and the manual validation scripts referenced in this appendix are openly available at <https://github.com/dbernhardt8/Extending-Escrow-Transactions-for-Heterogeneous-Resource-Management>. Each validation scenario in Table 15 corresponds to a script under the repository `tests/` directory. The scenario-to-script mapping is given in Table 16.

Core Schema Listing

The three tables below carry the central contributions: the escrow capacity counter (`Capacity`), the append-only journal (`AllocationJournal`), and the active-binding registry (`ActiveAllocation`). They are reproduced from `sql/1_database_model.sql` in the repository. Ancillary tables, sequences, and derived views are omitted.

Escrow capacity counter. The `Capacity` table holds the lock-free escrow counter. The `active_count` column is declared `RESERVABLE` so concurrent reservations update it without row locks.

```
-- Capacity: lock-free escrow counter (Oracle RESERVABLE column)
CREATE TABLE Capacity(
  id NUMBER PRIMARY KEY,
  context_id NUMBER NOT NULL,
  category_id NUMBER NOT NULL,
  total_capacity NUMBER NOT NULL CONSTRAINT check_total_capacity CHECK (
    total_capacity >= 0),
  active_count NUMBER RESERVABLE DEFAULT 0 NOT NULL CONSTRAINT
    check_active_count CHECK (active_count >= 0),
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
  metadata CLOB CHECK (metadata IS JSON),
  FOREIGN KEY (context_id) REFERENCES AllocationContext(id),
  FOREIGN KEY (category_id) REFERENCES ResourceCategory(id),
  UNIQUE(context_id, category_id),
  CONSTRAINT chk_capacity_valid CHECK (active_count <= total_capacity)
  DEFERRABLE INITIALLY DEFERRED
);
```

Append-only allocation journal. The AllocationJournal table records every lifecycle transition. It is list-partitioned by context_id and enforces the blocked-status user rule through a check constraint.

```
-- AllocationJournal: immutable append-only log, list-partitioned by context
CREATE TABLE AllocationJournal(
  id NUMBER PRIMARY KEY,
  context_id NUMBER NOT NULL,
  category_id NUMBER NOT NULL,
  user_id NUMBER,
  resource_instance_id NUMBER,
  status VARCHAR2(20) NOT NULL,
  entry_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  metadata CLOB CHECK (metadata IS JSON),
  CONSTRAINT fk_journal_context FOREIGN KEY (context_id) REFERENCES
  AllocationContext(id),
  CONSTRAINT fk_journal_category FOREIGN KEY (category_id) REFERENCES
  ResourceCategory(id),
  CONSTRAINT fk_journal_user FOREIGN KEY (user_id) REFERENCES Users(id),
  CONSTRAINT fk_journal_instance FOREIGN KEY (resource_instance_id) REFERENCES
  ResourceInstance(id),
  CONSTRAINT fk_journal_status FOREIGN KEY (status) REFERENCES ResourceStatus(
  name),
  CONSTRAINT chk_journal_user_blocked CHECK (
    (status = 'blocked' AND user_id IS NULL) OR
    (status <> 'blocked' AND user_id IS NOT NULL)
  )
)
PARTITION BY LIST (context_id)
(
  PARTITION journal_default VALUES (DEFAULT)
)
ENABLE ROW MOVEMENT;
```

Active-binding registry. The ActiveAllocation table holds at most one active binding per context and instance. The UNIQUE constraint enforces per-instance exclusivity.

```
-- ActiveAllocation: one active binding per (context, instance); UNIQUE enforces
  exclusivity
CREATE TABLE ActiveAllocation (
  context_id          NUMBER NOT NULL,
  resource_instance_id NUMBER NOT NULL,
  journal_id          NUMBER NOT NULL,
  CONSTRAINT uq_active_allocation UNIQUE (context_id, resource_instance_id),
  CONSTRAINT fk_aa_context  FOREIGN KEY (context_id)          REFERENCES
  AllocationContext(id),
  CONSTRAINT fk_aa_instance FOREIGN KEY (resource_instance_id) REFERENCES
  ResourceInstance(id)
);
```

Schema-to-Formal-Model Mapping

Table 12 links formal symbols from Section 3 to implementation tables and the invariants they enforce. See also Figure 2 in Section 4.

Table 12: Mapping from formal model to implementation schema

Formal	Implementation	Enforces
$\mathcal{T}$	ResourceCategory	Type-based requests
$\mathcal{R}, \text{type}(r)$	ResourceInstance	Per-instance identity
$\mathcal{A}, K(a, \tau)$	ResourceAsset, AssetCapacity	Contained pool size
Γ, I_γ	AllocationContext	Temporal context
$\mathcal{J}$	AllocationJournal	Append-only audit
$\mathcal{L}$	ActiveAllocation	Within-context exclusivity (UNIQUE)
Escrow hold	Capacity.active_count (RESERVABLE)	Pool admission (chk_capacity_valid)
$\mathcal{S}, c(\tau_i, \tau_j)$	CategorySubstitution	Substitution policy

Public API Surface

Table 13 groups representative procedures from the `ResourceManagement` package by concern. It is not an exhaustive listing. Workflow authors call `ResourceManagement` for reservations and lifecycle transitions. Context setup for comprehensive blocking uses `ResourceManagement.Data.AddAllocationContext` directly, as validated in scenario S4 (Maintenance overlap blocking).

Per-instance `BlockResource` and `UnblockResource` exist in the public package for ad-hoc single-instance blocks. Comprehensive cross-context blocking is driven by context creation with typed metadata (Section 4.5), not by `BlockResource`. Flight-wide cancellation uses `CancelFlight`, which loops `CancelReservation` and marks the context metadata as cancelled (Section 4.8). See scenario S4 and Table 15 (Appendix: Validation Scenario Index).

Table 13: Grouped public API surface (representative procedures)

Group	Representative procedures	Thesis section
Reservation	<code>MakeReservation</code> ,	Section 4.7
Substitution	<code>MakeReservationWithAlternative</code>	Section 4.9
Shared mode	<code>AllocateResourceDirect</code> ,	Section 5
	<code>CheckResourceTimeConflict</code>	
Lifecycle	<code>ConfirmReservation</code> , <code>CheckInUser</code> , <code>BoardUser</code> , ...	Section 4.8
Recovery	<code>CancelReservation</code> ,	Section 4.8
	<code>CompensateJournalEntry</code> (internal)	
Saga orchestration	<code>CancelFlight</code>	Section 4.8
Comprehensive blocking	<code>AddAllocationContext</code> with maintenance metadata (Path A), <code>AddAllocationContext</code> with normal operational metadata (Path B)	Section 4.5

Implementation Pseudocode

The following sketches use Oracle-style transaction blocks. The flagship walkthrough follows the airline seat-hold path from Section 5 (flight LH123, seat 12A). Package procedures may extend the escrow buffer and write the journal in either order inside the same savepoint; the invariant is that a failed buffer implies no journal row, and a failed journal after a successful buffer triggers compensation.

MakeReservationByInstanceId (seat hold). Pool admission, per-instance exclusivity, and the queue-backed confirm/cancel hold window (Sections 4.7 and 4.7; scenarios V1 and V3).

```
BEGIN
  SAVEPOINT seat_hold;

  -- Passenger selected seat 12A on flight LH123 (admission checks omitted)

  -- Hold one pool seat via escrow buffer (RESERVABLE: no row lock yet)
  UPDATE Capacity
    SET active_count = active_count + 1
    WHERE context = 'LH123' AND category = 'Economy';
  -- Reject immediately if held capacity would exceed pool size
  -- (this session's buffer + peers' in-flight buffers + committed base)
  EXECUTE IMMEDIATE 'SET CONSTRAINTS chk_capacity_valid IMMEDIATE';

  -- Bind seat 12A exclusively; journal row commits autonomously
  AddAllocationJournal(context => 'LH123', instance => '12A',
    status => 'reserved', user => passenger_id);

  -- Same session blocks until workflow sends CONFIRM/CANCEL or timeout expires
  msg := BlockingDequeue(correlation_id(journal_id), timeout => 5 minutes);

  IF msg = 'CONFIRM' THEN
    -- Forward lifecycle step
    AddAllocationJournal(..., status => 'confirmed');
    COMMIT; -- merges escrow buffer into committed counter base
  ELSE -- CANCEL message or timeout
    -- Neutralise reserved binding; append cancelled entry
    AddAllocationJournal(..., status => 'cancelled',
      metadata => {reason: 'cancel' | 'timeout'});
    ROLLBACK; -- discards uncommitted escrow buffer
  END IF;

  EXCEPTION
    WHEN pool_exhausted THEN
      ROLLBACK; -- ORA-20001: no journal row for this attempt
    WHEN exclusive THEN
      ROLLBACK TO seat_hold;
      CompensateJournalEntry(..., 'cancelled', 'retry_conflict'); -- ORA-20910
  END;
```

AddAllocationJournal. Autonomous journal insert with ActiveAllocation maintenance (Section 4.5).

```
BEGIN
  PRAGMA AUTONOMOUS_TRANSACTION;

  -- Append immutable lifecycle transition (audit trail)
  journal_id := NEXT sequence value;
  INSERT INTO AllocationJournal(journal_id, ..., status, metadata);

  IF instance IS NOT NULL THEN
    IF status IN (cancelled, completed) THEN
      -- Release exclusivity lock so the seat can be re-allocated
      DELETE FROM ActiveAllocation WHERE context AND instance;
      IF no row deleted THEN
        ROLLBACK;
        RAISE exclusive_removed; -- ORA-20912
      END IF;
    ELSIF status IN (reserved, blocked) THEN
      -- Claim lock: one active binding per (context, instance)
      BEGIN
        INSERT INTO ActiveAllocation(context, instance, journal_id);
      EXCEPTION
        WHEN unique_violation THEN
          ROLLBACK;
          RAISE exclusive; -- ORA-20910
        END;
    ELSE
      -- Forward transition: update binding to latest journal entry
      UPDATE ActiveAllocation SET journal_id WHERE context AND instance;
      IF no row updated THEN
        ROLLBACK;
        RAISE binding_absent; -- ORA-20911
      END IF;
    END IF;
  END IF;

  COMMIT; -- persists independently of caller transaction
END;
```

CompensateJournalEntry. Saga compensation via append-only inverse journal row (Section 4.8).

```
BEGIN
  -- Read the committed entry that must be neutralised
  READ original row FROM AllocationJournal WHERE id = original_journal_id;

  -- Append semantic inverse (e.g. reserved -> cancelled), not a replay
  AddAllocationJournal(same context/category/user/instance,
                      status = revert_status,
                      metadata = {compensation_reason: reason,
                                   original_journal_id: original_journal_id});
EXCEPTION
  WHEN OTHERS THEN
    LOG warning; -- caller may already be in exception handler
END;
```

CancelFlight (mass cancellation saga). Flight-wide release of active bindings under one savepoint (Section 4.8). Context metadata closes the flight to new admissions; only existing bindings receive cancelled journal rows.

```
BEGIN
  SAVEPOINT flight_cancel;

  FOR EACH active binding b ON context flight DO
    CancelReservation(b.journal_id, metadata = {source: flight_cancellation, ...
    });
    -- autonomous cancelled row; pool decrement in main tx escrow buffer
    track b's new journal id for compensation list;
  END LOOP;

  UPDATE AllocationContext
    SET metadata = {status: cancelled, reason, bindings_released: count}
    WHERE context = flight;

  COMMIT; -- merge escrow decrements + metadata; cancel rows already durable
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK TO flight_cancel;
    FOR EACH id IN compensation list DO
      CompensateJournalEntry(id, revert_status, reason);
    END LOOP;
    RAISE;
END;
```

Context creation and comprehensive blocking. Operational context setup, overlap guard, and Path A/Path B blocking (Section 4.5; scenario S4).

```
-- Operational context creation (e.g. flight LH123)
BEGIN
  IF asset inactive THEN RAISE inactive_asset; -- ORA-20301
  IF overlapping operational context on same asset THEN RAISE overlap; -- ORA
    -20302
  IF asset has no AssetCapacity rows THEN RAISE unconfigured; -- ORA-20303

  INSERT INTO AllocationContext(asset, identifier, start, end, metadata);
  FOR EACH pool category on asset DO
    INSERT INTO Capacity(context, category, total_capacity, active_count = 0);
  END LOOP;

  -- Path B: preemptive blocked rows for instances under maintenance
  IF metadata.context_type = normal THEN
    FOR EACH instance under overlapping maintenance contexts DO
      AddAllocationJournal(..., status = blocked, user = NULL);
    END LOOP;
  END IF;
END;

-- Path A: retroactive blocking after maintenance context is created
AddAllocationContext(asset, maint_identifier, start_m, end_m,
  metadata = {context_type: maintenance, resource_instance_ids: [M1, M2]});
FOR EACH overlapping operational context gamma DO
  FOR EACH instance IN [M1, M2] with active binding in gamma DO
    AddAllocationJournal(gamma, instance, status = blocked); -- Path A
    retroactive
  END LOOP;
END LOOP;

-- Overlap guard (scenario S4): reject overlapping operational contexts
TRY AddAllocationContext(asset, flight_B, start_b, end_b,
  metadata = {context_type: normal})
ON overlap REJECT; -- ORA-20302
```

Application Error Catalogue

Table 14 lists application errors referenced in the pseudocode above and maps them to validation scenarios from Section 4.10 and Table 15 (Appendix: Validation Scenario Index). It is not an exhaustive inventory of every `RAISE_APPLICATION_ERROR` in the package bodies (lifecycle, substitution, and administrative paths define additional codes).

Table 14: Application errors and validated scenarios

Code / signal	Meaning	Validated scenario
ORA-20001	Pool exhausted / high demand	Concurrent pool admission and compensation (V5)
ORA-20301	Asset inactive	Context setup precondition (not scenario-indexed)
ORA-20302	Overlapping operational context on same asset	Maintenance overlap blocking (S4)
ORA-20303	Asset has no <code>AssetCapacity</code> rows	Context setup precondition (not scenario-indexed)
ORA-20604	Instance already reserved in context (pre-check)	Double-booking rejection (V2)
ORA-20703	Shared resource interval overlap	Shared-resource interval conflict (V4)
ORA-20910	<code>ActiveAllocation</code> unique violation (race)	Double-booking rejection (V2); concurrent pool admission (V5)
ORA-20911	<code>ActiveAllocation</code> row missing on forward transition	Internal integrity under concurrency (V5 compensation path)
ORA-20912	<code>ActiveAllocation</code> already removed on cancel/complete	Internal integrity under concurrent cancel

Validation Scenario Index

Table 15 lists all manual validation scenarios. Scenarios V1–V5 appear in Section 4.10. Scenario S4 is the reference for Path A and Path B comprehensive blocking.

Table 15: Validation scenario index

ID	Scenario	Purpose
V1	Instance reservation and confirmation	Reserve a named seat. Confirm via event queue. Verify journal lifecycle and capacity commit
V2	Double-booking rejection	Second session targets the same seat. Exclusivity on $\mathcal{L}$ rejects before hold completes
V3	Hold-timeout auto-cancellation	No confirm within timeout. cancelled rows carry timeout metadata. Capacity is restored
V4	Shared-resource interval conflict	Overlapping meeting on a shared resource. Temporal exclusivity rejects the overlap
V5	Concurrent pool admission and compensation	Two sessions race for last pool seats. One wins. Loser compensates or receives a pool exhausted error
S1	Category pool reservation	Reserve N seats from a category pool without naming instances
S2	Explicit substitution offer	Offer named alternative categories when requested type is insufficient
S3	Partial substitution offer	Hold partial original quantity plus substitute categories under one offer group
S4	Maintenance overlap blocking	Path A retroactive and Path B preemptive blocked propagation. Includes overlap guard

Table 16 maps each scenario to the corresponding validation script in the repository (paths are relative to the `tests/` directory). The `manual/` scripts embed explicit pass/fail assertions, whereas the `demo/` scripts are interactive walkthroughs of the same paths. Multi-session scenarios and those exercising both an interactive and an assertion-based script list more than one file.

Table 16: Validation scenario to repository script mapping (paths relative to **tests/**)

ID	Script(s)
V1	demo/demo.02_request_by_instance.sql
V2	manual/ResourceRequest/ MANUAL_MakeReservationByInstanceId_DoubleBook.sql demo/demo.06_doublebook.sql
V3	demo/demo.08_aq_timeout.sql
V4	demo/demo.07_time_conflict.sql
V5	demo/demo.03_compensation_concurrent.sql manual/concurrency/MANUAL_Concurrent_ActiveAllocation.sql
S1	demo/demo.01_request_by_category.sql
S2	demo/demo.04_explicit_alternatives.sql
S3	demo/demo.05_partial_substitution.sql
S4	demo/demo.09_maintenance_overlap_contained.sql
Cancel- flight saga	manual/Administrative/MANUAL_CancelFlight.sql

AI Usage Disclosure

This thesis was developed with assistance from large language models (LLMs). The following table documents the scope, model, and purpose of AI usage. All AI-assisted content was reviewed, verified, and where necessary modified by the author.

Scope	Model	Purpose
Global editorial pass	Anthropic Claude Opus 4.5	Consistency checks, reading-flow improvements, terminology standardization, and editorial discussion support
ToC	OpenAI ChatGPT (web interface)	Initial structure and section outline
Section 1	Anthropic Claude Opus 4.5	Terminology fixes
Section 2	OpenAI ChatGPT (web interface)	Candidate related-work suggestions. Retained references were independently retrieved, read, and verified by the author before citation
Section 3	Anthropic Claude Opus 4.5	Drafting mathematical $\text{\LaTeX}$ formulas from formal definitions, cross-referencing, subsection structuring
Section 4	Anthropic Claude Opus 4.5	Subsection structuring, TikZ diagram drafts, transaction-strategy prose revision, table drafts, and $\text{\LaTeX}$ formatting
Section 5	Anthropic Claude Opus 4.5	TikZ diagram drafts, table drafts, cross-referencing, figure corrections, and prose revision for workflow integration
Notation table	Anthropic Claude Opus 4.5	Generating $\text{\LaTeX}$ table structure
Implementation appendix	Anthropic Claude Opus 4.5	Translating PL/SQL procedures from the package bodies into pseudocode sketches.
Comprehensive evaluation	Anthropic Claude Opus 4.5	Structured self-review prompts and critique drafts across multiple assessment dimensions (problem-solution alignment, structure, formal consistency, cross-referencing, readability, technical soundness, etc.)

Table 17: AI usage disclosure for this thesis

Provenance and verification: AI support was used during drafting and revision phases of this thesis (January–June 2026) through conversational interfaces (Cursor and ChatGPT). Model names and versions are reported according to labels shown in those interfaces at time of use. All factual claims, formal definitions, references, and final wording decisions were validated and approved by the author.

Statement: The research ideas, formal model, implementation, validation scenarios, and analysis are the author’s own work. Large language models assisted with L^AT_EX structure, diagram and table drafts, prose revision, consistency editing, and structured self-review prompts. Every assisted passage was checked against the implementation and revised by the author before inclusion.

Selbständigkeitserklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbständig verfasst und noch nicht für andere Prüfungen eingereicht habe. Sämtliche Quellen einschließlich Internetquellen, die unverändert oder abgewandelt wiedergegeben werden, insbesondere Quellen für Texte, Grafiken, Tabellen und Bilder, sind als solche kenntlich gemacht. Mir ist bekannt, dass bei Verstößen gegen diese Grundsätze ein Verfahren wegen Täuschungsversuchs bzw. Täuschung eingeleitet wird.

Berlin, den July 7, 2026

.....